

# 人工智能技术

## Artificial Intelligence

——人工智能: 经典智能+智能计算+智能学习

AI: Classical Intelligence + Computing Intelligence + Learning Intelligence

王鸿鹏, 王润花, 韩明静,

许丽, 靖智博

南开大学人工智能学院

hanmj@mail.nankai.edu.cn





# 第三部分 智能学习

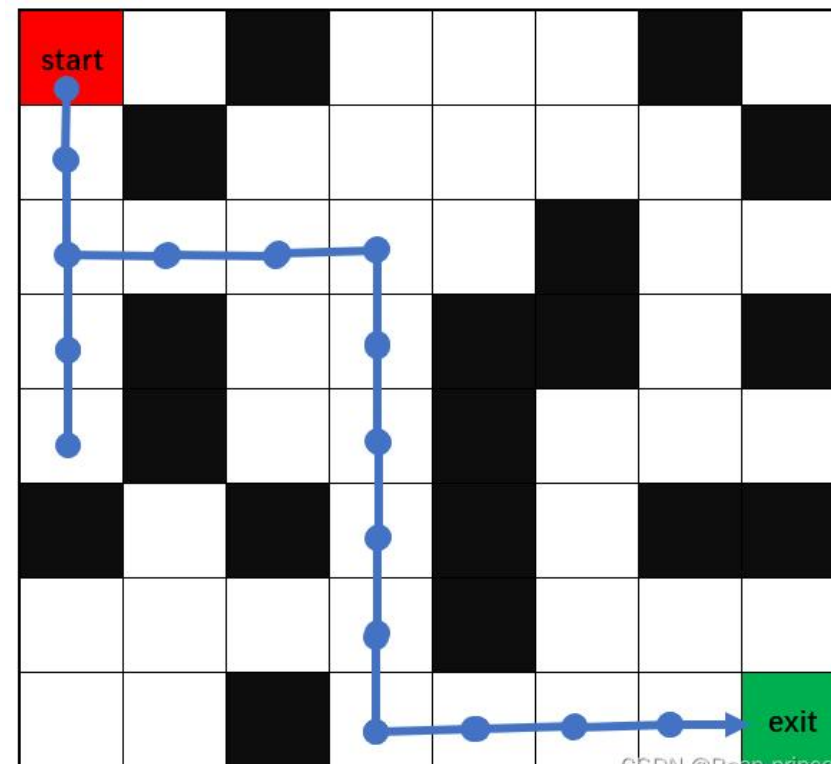
## Learning Intelligence

# 研究背景

**逻辑智能：**人类手工编码规则，用逻辑推理实现“智能”  
(专家系统)

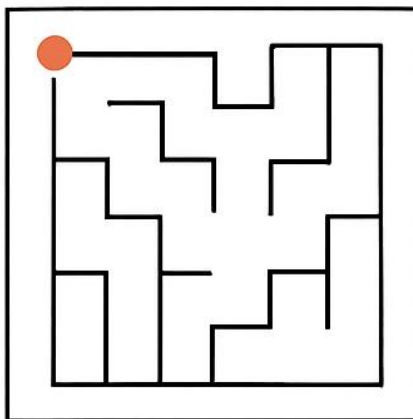
**计算智能：**强调仿生与启发式搜索，不依赖严格的逻辑规则  
(模糊逻辑、进化算法、群智能，早期人工神经网络)

**学习智能：**数据驱动，通过机器学习从数据中自动抽取规律  
(深度学习、强化学习)



# 机器人在迷宫中寻找出口

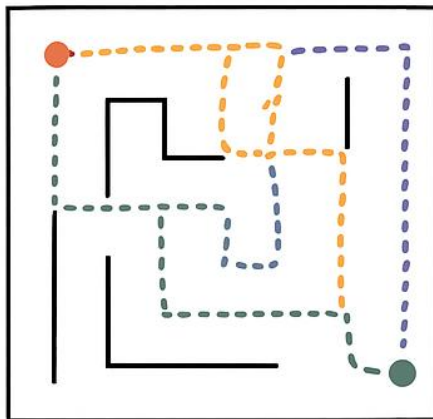
## 逻辑智能



### 基于规则的路径规划

```
while not at_goal
    if front_blocked
        turn_right()
    else if right_open
        move_right()
    move_forward()
```

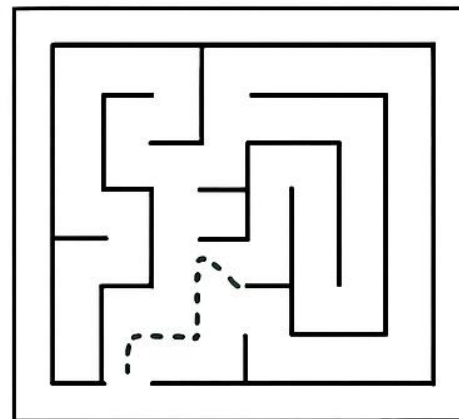
## 计算智能



### 仿生/启发式搜索

```
for lteration in range(max_
    ver_iter)"
    for each ant
        path = explore_maze()
        deposit_pheromone(peth
        evaporate
        pheromone()
}
```

## 学习智能



### 数据驱动的策路学习

```
for episode in range(N)
    s = start
    while not at_goal
        a = select_action(Q,
        s', r = step(s, a)
        Q[s,a] = Q[s,a] +  $\alpha$ 
        (r +  $\gamma$ *max(Q[s']) - Q1
```

# 定义

## 学习

- 西蒙(Simon): 学习就是**系统**在不断**重复的工作中**对本身能力的**增强**或者**改进**, 使得系统在下次执行同样任务或**类似**任务时, 会比现在做得更好或效率更高。

Learning denotes **changes** in the system that are adaptive in the sense that they enable the system to do the task or tasks drawn from the same population more efficiently and more effectively the next time.

by Herbert Simon

学习的本质是系统性能的自我改进



赫伯特·西蒙  
(Herbert Simon)

# 定义

## 学习

- 维纳(Wiener, 1965): 一个具有生存能力的动物在它的一生中能够被其经受的环境所改造。一个能够繁殖后代的动物至少能够生产出与自身相似的动物(后代), 即使这种相似可能随着时间变化。如果这种变化是自我可遗传的, 那么就存在一种能受自然选择影响的物质。如果该变化是以行为形式出现的, 并假定这种行为是无害的, 那么这种变化就会世代相传下去。这种从一代至下一代的变化形式称为**种族学习**(racial learning)或**系统发育学习**(system growth learning), 而发生在特定个体上的这种行为变化或行为学习, 则称为**个体发育学习**(individual growth learning)

一个具有生存能力的系统（动物或机器）之所以能持续存在，是因为它能够被环境所塑造，即能在经验中改变自身的行为模式——**适应能力**

# 定义

## 学习

- 香农(C. Shannon, 1953): 假设 ①一个有机体或一部机器处在某类环境中, 或者同该环境有联系; ②对该环境存在一种“成功的”度量或“自适应”度量; ③这种度量在时间上是比较局部的, 也就是说, 人们能够用一个比有机体生命期短的时间来测试这种成功的度量。对于所考虑的环境, 如果这种全局的成功度量能够随时间而改善, 那么我们就说, 对于所选择的成功度量, 该有机体或机器正为适应这类环境而学习。

学习 = 环境交互 + 局部可评估目标 + 目标导向的性能提升

# 定义

## 学习

- 米切尔(Mitchell): 对于某类任务 $T$ 和性能度量 $P$ , 如果一个计算机程序在 $T$ 上以 $P$ 衡量的性能随着经验 $E$ 而自我完善, 那么就称这个计算机程序从经验 $E$ 中学习。

Machine Learning  $\langle T, P, E \rangle$

Any computer program that improves its performance  $P$  at some task  $T$  through experience  $E$ .

by Tom M. Mitchell

系统在特定任务上, 通过经验不断提高其性能



汤姆·米切尔  
(Tom M. Mitchell)



# Examples of the learning tasks:

---

- ✿ Playing checkers
  - ✦ T: Playing checkers
  - ✦ P: Percentage of games won against an arbitrary opponent
  - ✦ E: Playing practice games against itself
- ✿ Recognizing
  - ✦ T: Recognizing hand-written words
  - ✦ P: Percentage of words correctly classified
  - ✦ E: Database of human-labeled images of handwritten words
- ✿ Automatically driving
  - ✦ T: Driving on four-lane highways using vision sensors
  - ✦ P: Average distance traveled before a human-judged error
  - ✦ E: A sequence of images and steering commands recorded while observing a human driver.

# 定义

## 学习系统

- 学习系统(Learning system)是一个能够学习有关过程的未知信息，并用所学信息作为进一步决策或控制的经验，从而逐步改善过程性能的系统。
- 如果一个系统能够学习某一过程或环境的未知特征固有信息，并用所得经验进行估计、分类、决策和控制，使系统的品质得到改善，那么称该系统为学习系统。
- 学习系统是一个能在其运行过程中逐步获得过程及环境的非预知信息，积累经验，并在一定的评价标准下进行估值、分类、决策和不断改善系统品质的智能系统。

让系统具备经验驱动的自我改进能力

# 定义

## 机器学习

- 顾名思义，机器学习是研究如何使用机器来模拟人类活动的一门学科。
- 稍微严格一点的提法是：机器学习是一门研究机器获取新知识和新技能，并识别现有知识的学问。
- 综合上述两定义，可给出如下定义：

机器学习是研究机器模拟人类的学习活动，获取知识和技能的理论和方法，以改善系统性能的学科。

# 机器学习的发展史

- ❁ 机器学习的发展分为4个时期
  - ❏ 热烈时期（50年代中叶到60年代中叶）
    - 神经元模型研究
    - 罗森勃拉特1957年提出的感知器模型
  - ❏ 冷静时期（60年代中叶至70年代中叶）
    - 符号概念获取
    - 模拟人类的概念学习过程
  - ❏ 复兴时期（70年代中叶至80年代中叶）
    - 知识强化学习
    - 与各种实际应用相结合，尤其是专家系统在知识获取方面
  - ❏ 最新阶段（1986年至今）
    - 连接学习和混合型学习
    - 符号学习和连接学习结合

# 机器学习的发展史

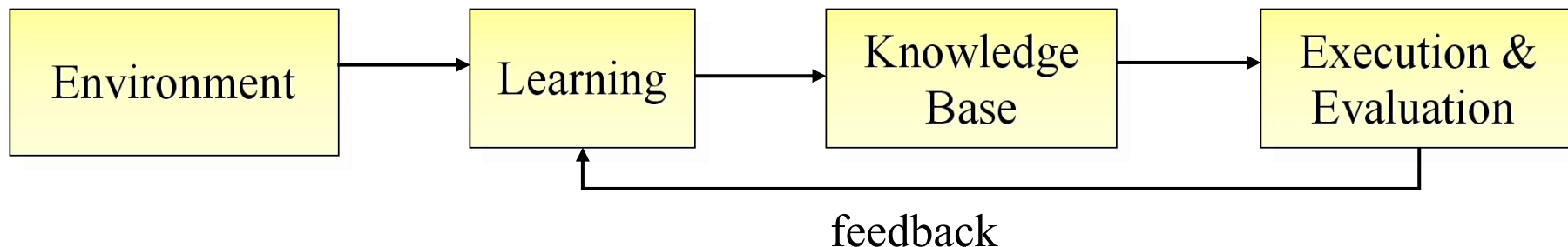
---

- ❁ 机器学习进入新阶段的表现
  - ❁ 机器学习已成为新的学科并在高校形成课程。
  - ❁ 综合各种学习方法。
  - ❁ 机器学习与人工智能问题的统一性观点正在形成。
  - ❁ 各种学习方法的应用范围不断扩大。
  - ❁ 数据挖掘和知识发现的研究已形成热潮。
  - ❁ 与机器学习有关的学术活动空前活跃。

# 机器学习系统的基本结构

以西蒙的学习定义为出发点，建立起简单的学习模型：

环境向系统的学习部分提供某些信息，学习部分利用这些信息修改知识库，以增进系统执行部分完成任务的效能，执行部分根据知识库完成任务，同时把获得的信息反馈给学习部分。在具体的应用中，环境，知识库和执行部分决定了具体的工作内容，学习部分所需要解决的问题完全由上述3部分确定。



# 影响学习系统设计的要素

---

## ✿ 环境向系统提供的信息

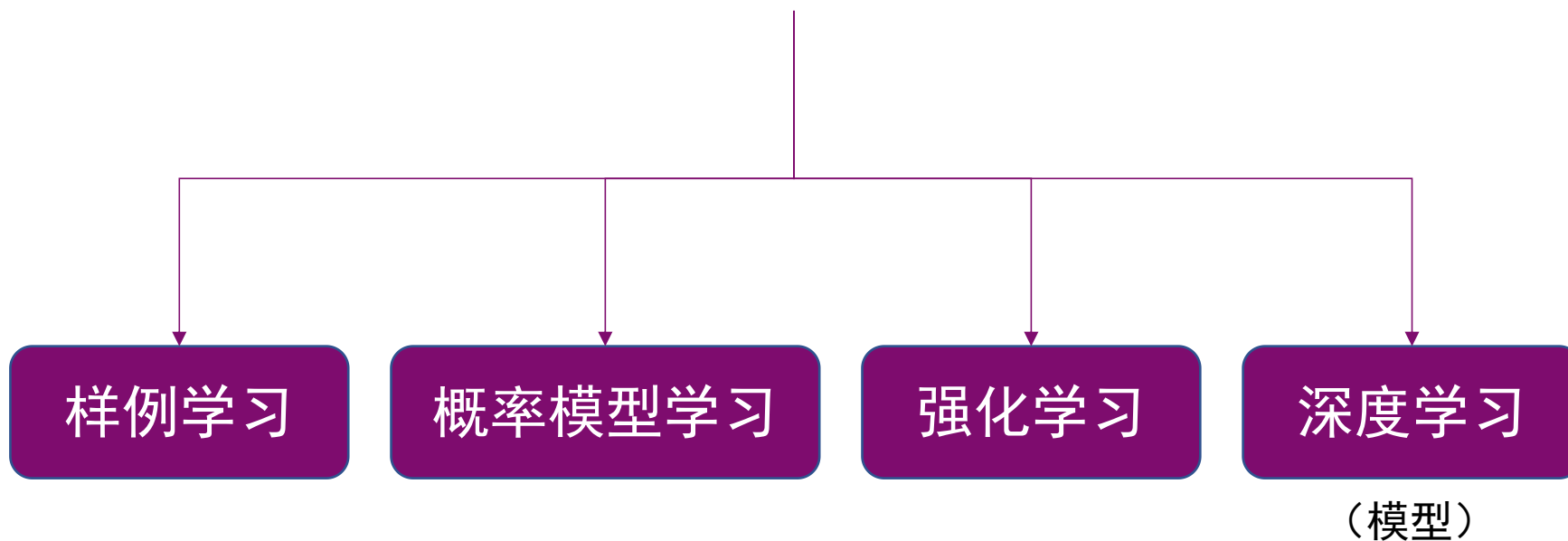
- ✦ 或者更具体地说是信息的质量
- ✦ 最重要因素

## ✿ 知识库

- ✦ 是影响学习系统设计的第二个因素
- ✦ 知识的表示有特征向量、一阶逻辑语句、产生式规则、语义网络和框架等多种形式

# 研究背景

**机器学习 = 学习方式 × 学习模型 × 学习目标**





# 样例学习

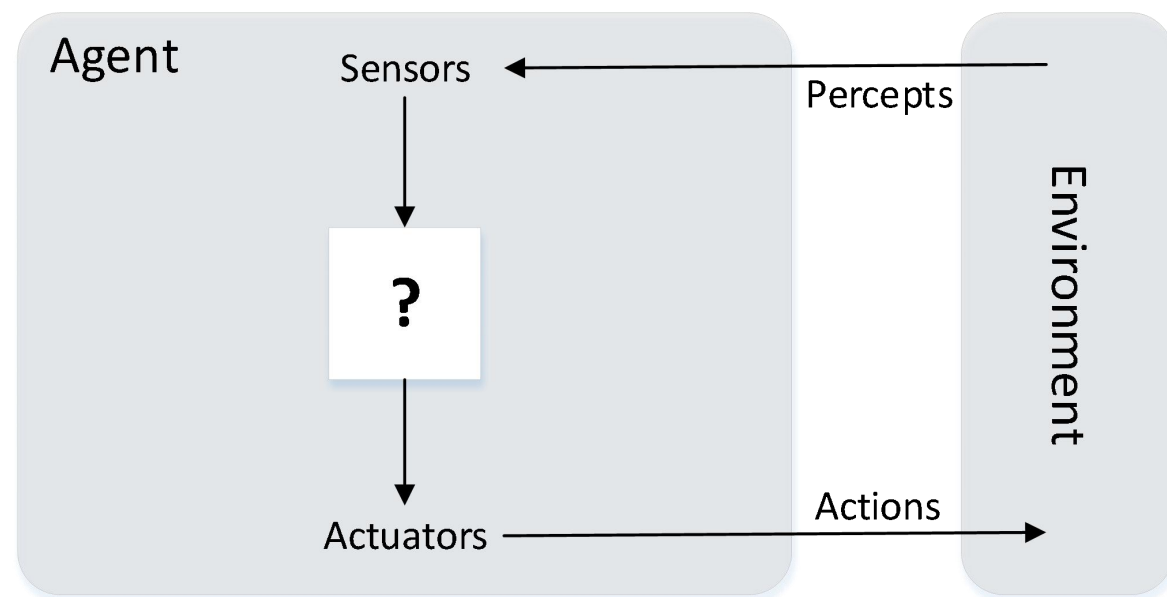
## LEARNING FROM EXAMPLES

描述智能体通过不断学习自己以往的经验从而改善自己的行为，并对未来进行预测的过程。

# 学习的形式

机器学习改进智能体及用于改进的技巧**取决于**:

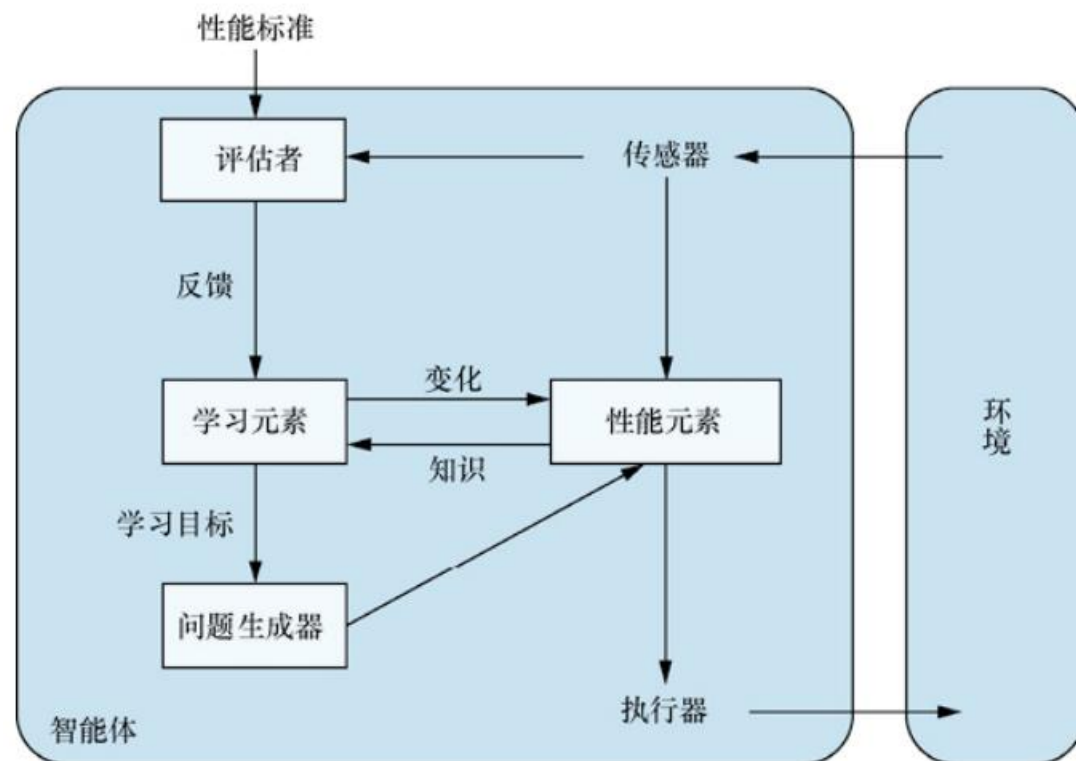
- 哪些**组件**可以被改进
- 智能体有哪些**先验知识**, 这将影响模型构建
- 有哪些**数据**, 以及关于这些数据的**反馈**



# 学习的形式

## 智能体组件：

- (1) 从当前状态条件到动作的直接映射；
- (2) 用于从感知序列推断世界相关性质的方法；
- (3) 关于世界演化方式的信息，以及关于智能体可以采取的可能动作所导致的结果的信息；
- (4) 表示状态意向的效用信息；
- (5) 表示动作意向的动作-价值信息；
- (6) 最希望达到的状态，即目标；
- (7) 问题生成器、评判标准和使系统得以改进的学习元素。



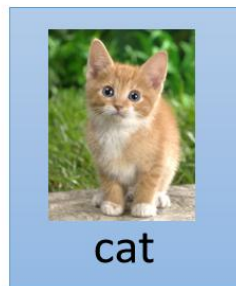
# 学习的形式

**智能体先验知识——智能体在学习前已具备的关于环境或任务的信息**

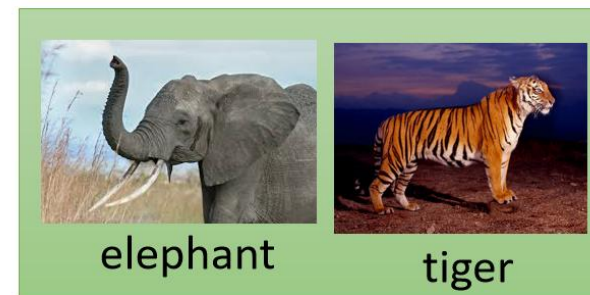
- **零先验：**智能体没有任何先验知识，完全依赖数据或环境交互进行学习
- **部分先验：**智能体拥有部分规则、约束或领域模型
- **迁移学习：**智能体把一个领域学到的知识迁移到另一个相关领域



Different domains, same task



Dog/Cat  
Classifier



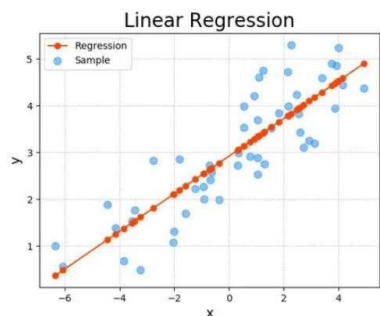
Similar domain, different tasks

# 学习的形式

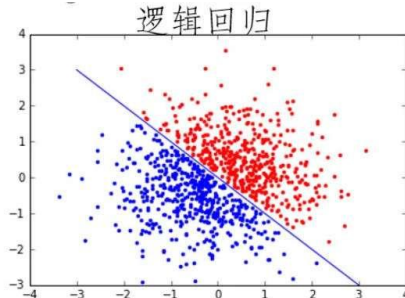
## 智能体数据:

- 归纳: 特殊 → 概括出一般
- 演绎: 一般 → 推出特殊

## 输入与输出



回归问题



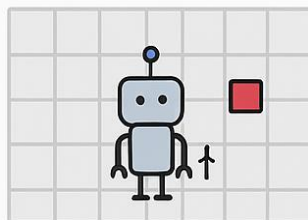
分类问题

## INPUT REPRESENTATION

### Factored Representation 因子化表示

The state is represented by a set of features or variables

- Structured, easy to use with numerical models
- Can capture relationships
- Requires defining the feature set

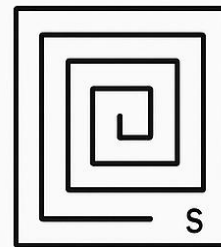


(x, y, direction, obstacle)

### Atomic Representation 原子表示

The state is treated as an indivisible whole symbol-

- Simple
- Suitable for small, finite state spaces



S17

### Relational Representation 关系表示

The state is represented by objects and relations

- Expressive
- Suitable for complex scenarios
- Harder to handle



piece(white, king, e1)  
piece(black, queen, d8)  
attacked(white, d5)

# 学习的形式

伴随输入有3种类型的**反馈**，决定了3种类型的学习：

- **监督学习** (supervised learning)：智能体观测到输入-输出对，并学习从输入到输出(标签, label)的一个函数映射
- **无监督学习** (unsupervised learning)：智能体从没有任何显式反馈的输入中学习模式。(eg, 聚类)
- **强化学习** (reinforcement learning)：智能体从一系列的强化——奖励与惩罚——中进行学习

# 监督学习

给定一个**训练集** (training set) 含有  $N$  个 “输入-输出” 对样例:

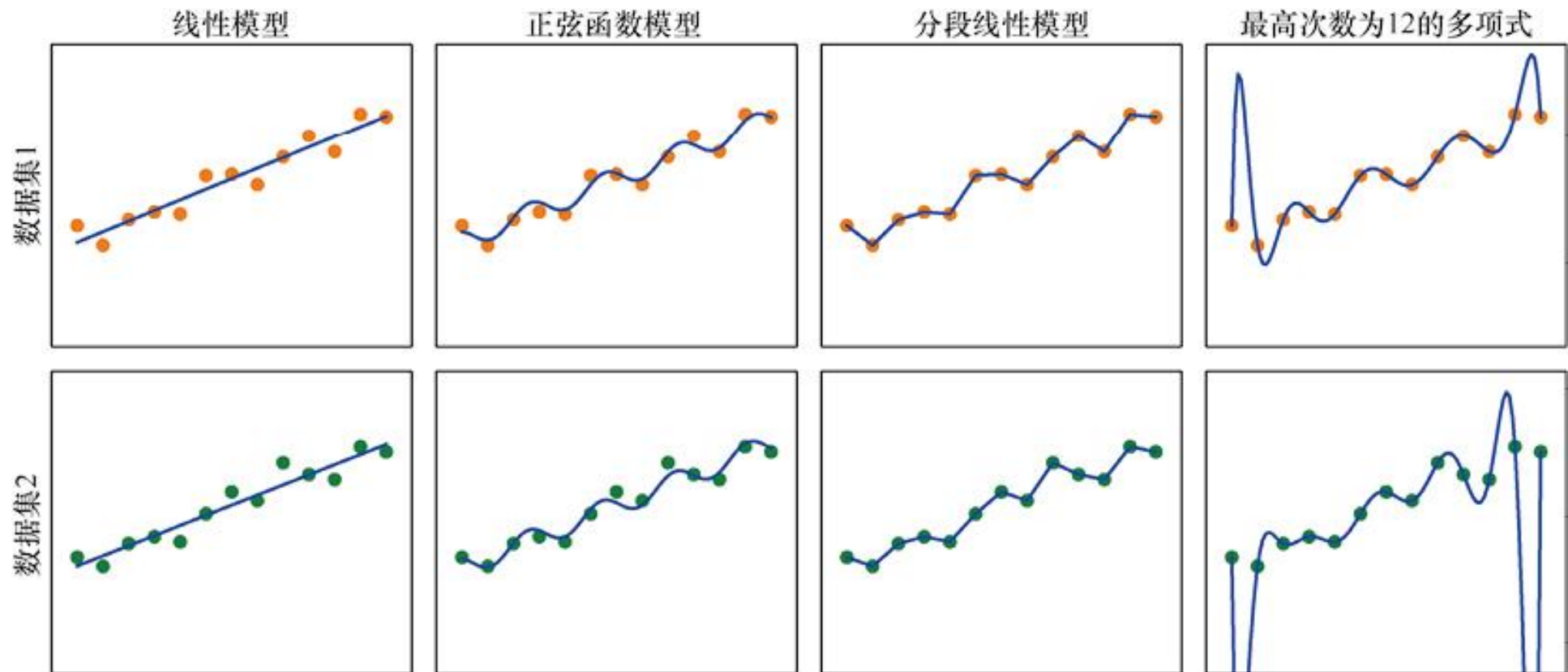
$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$$

其中每一对数据都由一个未知的函数  $y = f(x)$  生成, 寻找一个函数  $h$  来近似真实的函数  $f$ 。

- 函数  $h$  被称为关于世界的**假设** (hypothesis)。它取自一个**假设空间** (hypothesis space)  $H$ , 其中包含所有可能的函数。
- 也可称  $h$  是关于数据的**模型**, 它取自**模型类** (model class)  $H$ , 也可以说它取自**函数类** (function class) 中的一个**函数** (function)。我们称输出  $y_i$  为**真实数据** (ground truth)
- **一致性假设** (consistent hypothesis): 假设  $h$ , 对训练集中的任意一个  $x_i$ , 都有  $h(x_i) = y_i$ 。如果输出是连续值, 我们不能期望模型输出与真实数据精确匹配; 相反, 我们寄希望于寻找一个**最佳拟合函数** (best-fit function), 使得每一个  $h(x_i)$  与  $y_i$  非常接近
- 使用一个**测试集** (test set) —— 第二组样本数据对  $(x_i, y_i)$  —— 来评估假设。如果  $h$  准确地预测了测试集的输出, 我们称  $h$  具有很好的**泛化** (generalize) 能力。



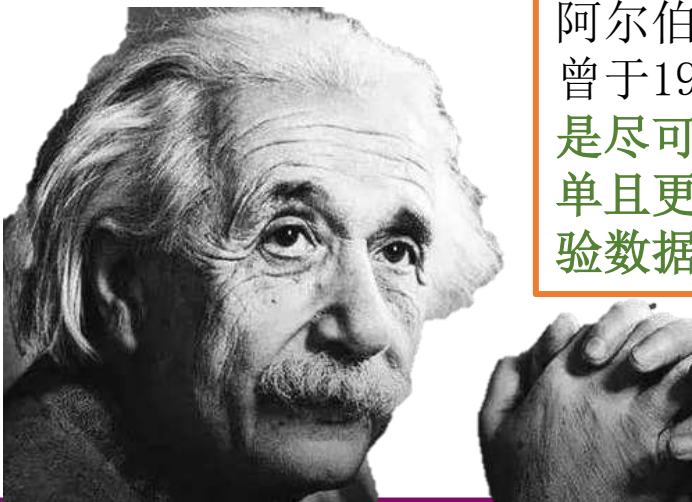
# 监督学习





# 监督学习

- **偏差** (bias) 是指 (不严格地) 在不同的训练集上, 假设所预测的值偏离期望值的平均趋势。偏差常常是由假设空间所施加的约束造成的。当一个假设不能找到数据中的模式时, 我们称它是**欠拟合** (underfitting) 的。
- **方差** (variance) 是指由训练数据波动而导致假设的变化量。当一个函数过于关注它用来训练的特定训练数据集, 进而导致它在没有见过的数据上表现较差时, 我们称该函数对数据集是**过拟合** (overfitting) 的。
- **偏差-方差权衡** (bias-variance tradeoff): 在更复杂、低偏差的能较好拟合训练集的假设与更简单、低方差的可能泛化得更好的假设中做出选择。



阿尔伯特·爱因斯坦 (Albert Einstein) 曾于1933年说过, “**任何理论的终极目标都是尽可能让不可削减的基本元素变得更加简单且更少, 但也不能放弃对任何一个单一经验数据的充分阐释**”。

**奥卡姆剃刀原则** (Ockham's razor):  
“**如无必要, 勿增实体**” ---- 14世纪英国哲学家奥卡姆的威廉 (William of Ockham)

# 监督学习

## “为什么学习系统倾向于选择简单假设？”

假设  $h^*$ （在数据集上  $h^*$  发生概率最大）：

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(h | data)$$

根据贝叶斯法则，上式等价于

$$h^* = \operatorname{argmax}_{h \in \mathcal{H}} P(data | h)P(h)$$

# 监督学习

## 示例问题：餐馆等位 (Restaurant Waiting)

- 这是一个关于是否在餐馆等待座位的决策问题。
- 在该问题中，输出变量  $y$  是一个布尔变量，称为 WillWait (是否等待)。
- 输入变量  $x$  是一个由十个属性值组成的向量，每个属性都有离散取值：
  1. Alternate (替代餐馆)：附近是否有合适的替代餐馆。
  2. Bar (酒吧区)：该餐馆是否有舒适的酒吧区可供等待。
  3. Fri/Sat (星期五或星期六)：在星期五或星期六时取值为真。
  4. Hungry (是否饥饿)：我们现在是否感到饥饿。
  5. Patrons (就餐人数)：餐馆内有多少人 (取值为 *None*、*Some* 或 *Full*)。
  6. Price (价格范围)：餐馆的价格水平 (取值为 \$、\$\$、\$\$\$)。
  7. Raining (是否下雨)：外面是否在下雨。
  8. Reservation (是否预订)：我们是否提前预订了座位。
  9. Type (餐馆类型)：餐馆的菜系类型 (取值为 *French*、*Italian*、*Thai* 或 *Burger*)。
  10. WaitEstimate (等待时间预估)：服务员估计的等待时间 (取值为 0-10、10-30、30-60 或 >60 分钟)。

# 监督学习

| 样例       | 输入属性             |            |                |               |                |              |                |                    |             |                     | 输出                    |
|----------|------------------|------------|----------------|---------------|----------------|--------------|----------------|--------------------|-------------|---------------------|-----------------------|
|          | <i>Alternate</i> | <i>Bar</i> | <i>Fri/Sat</i> | <i>Hungry</i> | <i>Patrons</i> | <i>Price</i> | <i>Raining</i> | <i>Reservation</i> | <i>Type</i> | <i>WaitEstimate</i> | <i>WillWait</i>       |
| $x_1$    | Yes              | No         | No             | Yes           | Some           | \$\$\$       | No             | Yes                | French      | 0 ~ 10              | $y_1 = \text{Yes}$    |
| $x_2$    | Yes              | No         | No             | Yes           | Full           | \$           | No             | No                 | Thai        | 30 ~ 60             | $y_2 = \text{No}$     |
| $x_3$    | No               | Yes        | No             | No            | Some           | \$           | No             | No                 | Burger      | 0 ~ 10              | $y_3 = \text{Yes}$    |
| $x_4$    | Yes              | No         | Yes            | Yes           | Full           | \$           | Yes            | No                 | Thai        | 10 ~ 30             | $y_4 = \text{Yes}$    |
| $x_5$    | Yes              | No         | Yes            | No            | Full           | \$\$\$       | No             | Yes                | French      | > 60                | $y_5 = \text{No}$     |
| $x_6$    | No               | Yes        | No             | Yes           | Some           | \$\$         | Yes            | Yes                | Italian     | 0 ~ 10              | $y_6 = \text{Yes}$    |
| $x_7$    | No               | Yes        | No             | No            | None           | \$           | Yes            | No                 | Burger      | 0 ~ 10              | $y_7 = \text{No}$     |
| $x_8$    | No               | No         | No             | Yes           | Some           | \$\$         | Yes            | Yes                | Thai        | 0 ~ 10              | $y_8 = \text{Yes}$    |
| $x_9$    | No               | Yes        | Yes            | No            | Full           | \$           | Yes            | No                 | Burger      | > 60                | $y_9 = \text{No}$     |
| $x_{10}$ | Yes              | Yes        | Yes            | Yes           | Full           | \$\$\$       | No             | Yes                | Italian     | 10 ~ 30             | $y_{10} = \text{No}$  |
| $x_{11}$ | No               | No         | No             | No            | None           | \$           | No             | No                 | Thai        | 0 ~ 10              | $y_{11} = \text{No}$  |
| $x_{12}$ | Yes              | Yes        | Yes            | Yes           | Full           | \$           | No             | No                 | Burger      | 30 ~ 60             | $y_{12} = \text{Yes}$ |

# 决策树学习

决策树 (Decision Tree) 是一种用于表示函数的结构, 它将一个属性值向量映射为一个单一的输出值——即一个“决策”。

- 决策树通过执行一系列测试来得出决策: 从树的根节点开始, 根据测试结果沿着相应的分支前进, 直到到达一个叶节点。

在决策树中:

- 每一个内部节点 (internal node) 对应于对某个输入属性取值的测试;
- 从该节点引出的分支 (branch) 用该属性的不同可能取值进行标记;
- 每一个叶节点 (leaf node) 指定了函数最终返回的输出值。

对于布尔型决策树 (Boolean Decision Tree), 它等价于一个逻辑表达式, 形式如下:

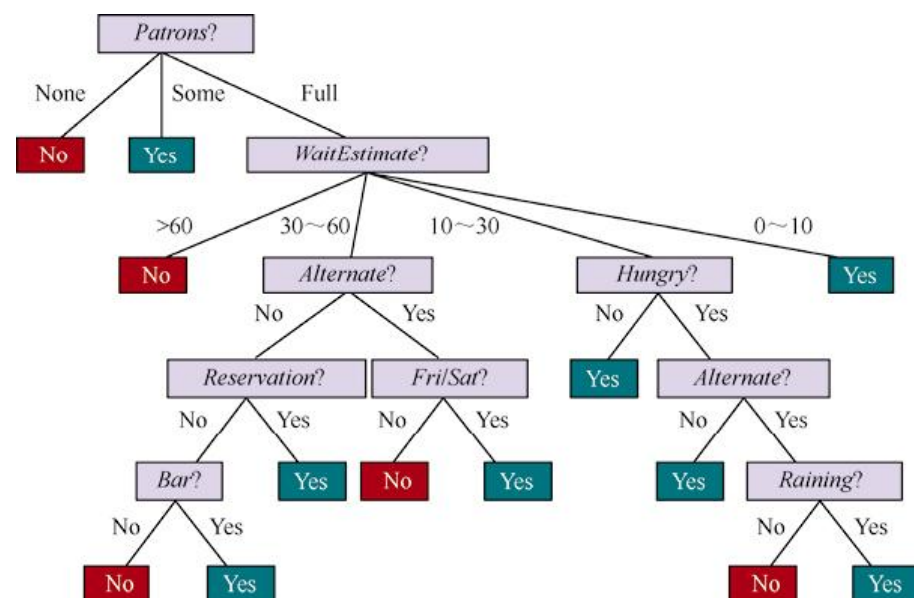
$$\text{Output} \Leftrightarrow (\text{Path1} \vee \text{Path2} \vee \dots)$$

其中, 每一个 Path 表示从根节点到某个“输出为真”的叶节点路径对应的一组属性取值条件的合取式 (AND 条件), 而整个树的输出是这些路径条件的析取 (OR)。

# 决策树学习

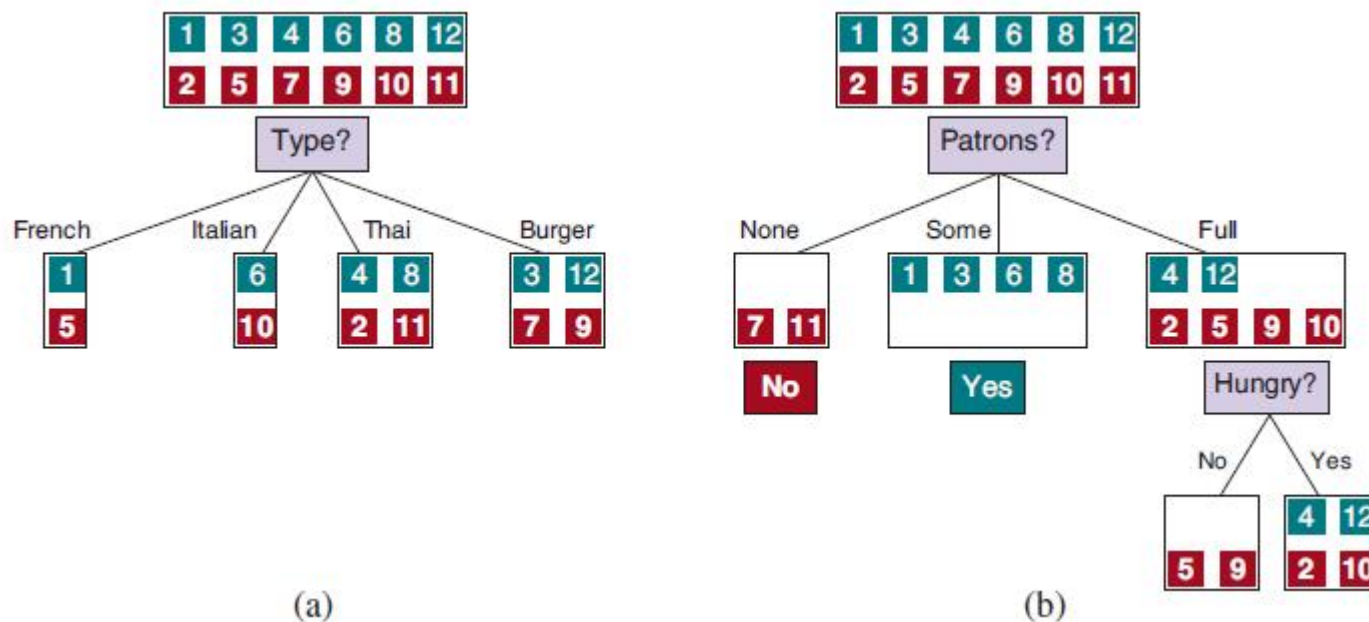
决策树 (decision tree) 表示一类函数：将属性值向量映射到单个输出值（即“决策”）。

| 样例       | 输入属性      |     |         |        |         |        |         |             |         |              | 输出                    |
|----------|-----------|-----|---------|--------|---------|--------|---------|-------------|---------|--------------|-----------------------|
|          | Alternate | Bar | Fri/Sat | Hungry | Patrons | Price  | Raining | Reservation | Type    | WaitEstimate | WillWait              |
| $x_1$    | Yes       | No  | No      | Yes    | Some    | \$\$\$ | No      | Yes         | French  | 0 ~ 10       | $y_1 = \text{Yes}$    |
| $x_2$    | Yes       | No  | No      | Yes    | Full    | \$     | No      | No          | Thai    | 30 ~ 60      | $y_2 = \text{No}$     |
| $x_3$    | No        | Yes | No      | No     | Some    | \$     | No      | No          | Burger  | 0 ~ 10       | $y_3 = \text{Yes}$    |
| $x_4$    | Yes       | No  | Yes     | Yes    | Full    | \$     | Yes     | No          | Thai    | 10 ~ 30      | $y_4 = \text{Yes}$    |
| $x_5$    | Yes       | No  | Yes     | No     | Full    | \$\$\$ | No      | Yes         | French  | > 60         | $y_5 = \text{No}$     |
| $x_6$    | No        | Yes | No      | Yes    | Some    | \$\$   | Yes     | Yes         | Italian | 0 ~ 10       | $y_6 = \text{Yes}$    |
| $x_7$    | No        | Yes | No      | No     | None    | \$     | Yes     | No          | Burger  | 0 ~ 10       | $y_7 = \text{No}$     |
| $x_8$    | No        | No  | No      | Yes    | Some    | \$\$   | Yes     | Yes         | Thai    | 0 ~ 10       | $y_8 = \text{Yes}$    |
| $x_9$    | No        | Yes | Yes     | No     | Full    | \$     | Yes     | No          | Burger  | > 60         | $y_9 = \text{No}$     |
| $x_{10}$ | Yes       | Yes | Yes     | Yes    | Full    | \$\$\$ | No      | Yes         | Italian | 10 ~ 30      | $y_{10} = \text{No}$  |
| $x_{11}$ | No        | No  | No      | No     | None    | \$     | No      | No          | Thai    | 0 ~ 10       | $y_{11} = \text{No}$  |
| $x_{12}$ | Yes       | Yes | Yes     | Yes    | Full    | \$     | No      | No          | Burger  | 30 ~ 60      | $y_{12} = \text{Yes}$ |





# 决策树学习



按属性划分样例：浅色方框表示正例，深色方框表示反例

- 当我们在属性 **Type**（餐厅类型）上进行划分时，各个分支中的正负样例仍然混杂，因此这种划分 **并没有让我们更接近区分正例与反例**。
- 在按 **Patrons** 划分之后，再使用属性 **Hungry**（是否饥饿）作为第二层划分，也是一个相对较好的选择。
- 相比之下，按属性 **Patrons**（顾客数量）进行划分效果很好，它能较好地区分出哪些情况会等待（WillWait = Yes）与不会等待（WillWait = No）的样例。

# 决策树学习

```
function LEARN-DECISION-TREE(examples, attributes, parent_examples) returns a tree
  if examples is empty then return PLURALITY-VALUE(parent_examples)
  else if all examples have the same classification then return the classification
  else if attributes is empty then return PLURALITY-VALUE(examples)
  else
     $A \leftarrow \operatorname{argmax}_{a \in \text{attributes}} \text{IMPORTANCE}(a, \text{examples})$ 
    tree  $\leftarrow$  a new decision tree with root test A
    for each value v of A do
      exs  $\leftarrow \{e : e \in \text{examples} \text{ and } e.A = v\}$ 
      subtree  $\leftarrow$  LEARN-DECISION-TREE(exs, attributes - A, examples)
      add a branch to tree with label (A = v) and subtree subtree
    return tree
```

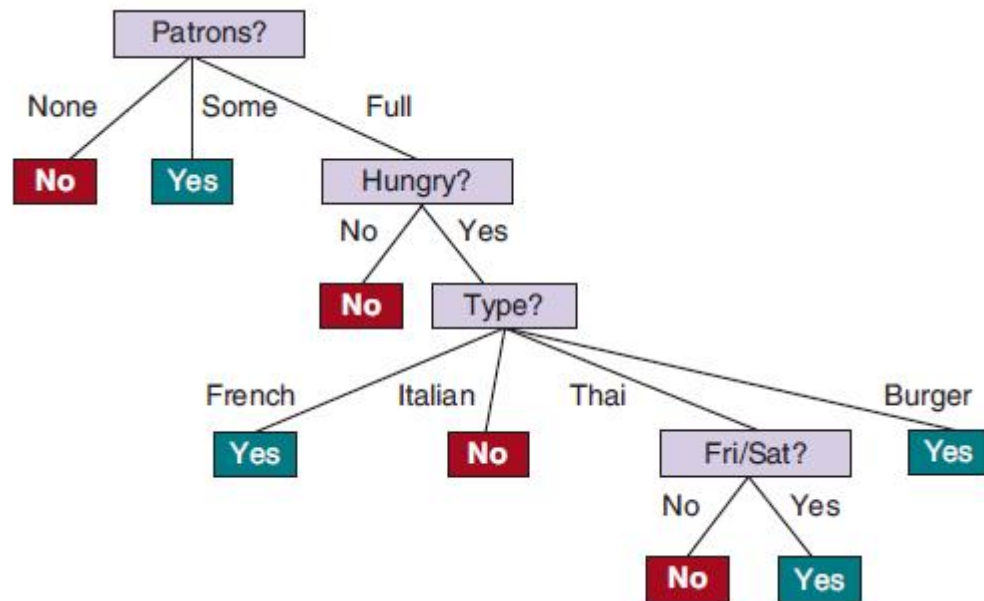
函数 PLURALITY-VALUE 在一组样例中选择最常见的输出值，如果出现并列，则随机打破平局

目标 (Aim)：找到一个与训练样例一致的较小的树。

基本思想 (Idea)：递归地选择“最重要的属性 (most significant attribute)”作为 (子) 树的根节点。

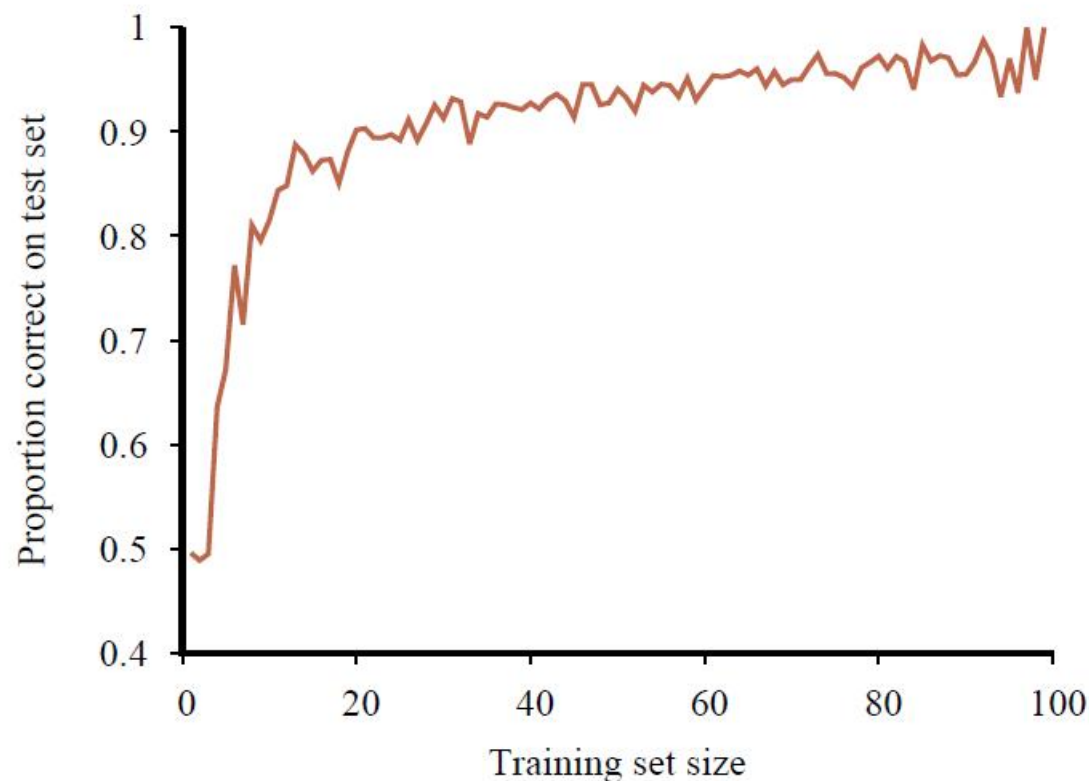


# 决策树学习



从 12 个训练样例中生成的决策树

# 决策树学习



该图展示了在餐厅决策问题 (restaurant domain) 上, 使用 100 个随机生成样例 时, 决策树学习算法的学习曲线 (learning curve) 。

每一个数据点都是在 20 次独立实验 (trials) 中取平均得到的结果, 反映了随着训练样本数量增加, 模型在测试集上的性能 (准确率或错误率) 的变化趋势

# 决策树学习

## 选择测试属性：信息增益

熵是随机变量不确定性的度量；信息量越多，熵越小

$$H(V) = \sum_K P(v_k) \log_2 \frac{1}{P(v_k)} = -\sum_k P(v_k) \log_2 P(v_k)$$

如果一个训练集包含

个正样例和n个负样例，则在整个集合上的输出变量的熵为：

$$H(\text{Output}) = B\left(\frac{p}{p+n}\right)$$

测试属性A后剩余的熵的期望

$$\text{Remainder}(A) = \sum_{k=1}^d \frac{p_k + n_k}{p+n} B\left(\frac{p_k}{p_k + n_k}\right)$$

通过测试属性A获得的信息增益（information gain）定义为熵减少的期望值

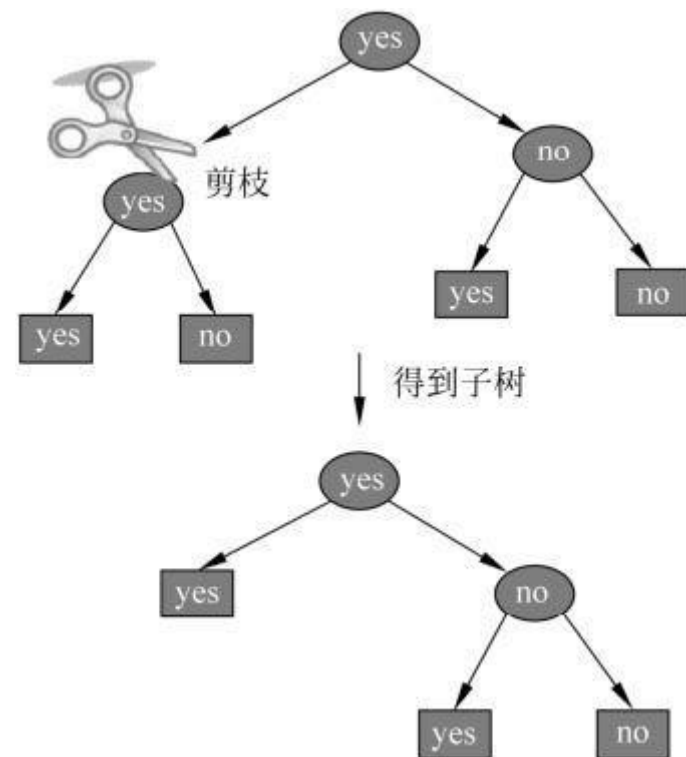
$$\text{Gain}(A) = B\left(\frac{p}{p+n}\right) - \text{Remainder}(A)$$

决策树剪枝：通过删去不明显相关的节点来减轻过拟合

# 决策树学习

## 泛化与过拟合

- ❖ 决策树剪枝有助于防止过拟合
  - 通过删除那些不明显相关的节点来实现
- ❖ 在决定是否基于某个属性进行分裂时，需要多大的增益？
  - 使用显著性检验
    1. 从零假设开始
    2. 计算数据偏离“完全不存在模式”的程度
    3. 偏离程度如果非常不可能发生（概率  $\leq 5\%$ ），则认为有统计显著性
- ❖ 假设一个节点包含  $p$  个正例和  $n$  个负例，期望数量为  $p^k$  和  $n^k$
- ❖ 测量偏差 & 总偏差



$$\hat{p}_k = p \times \frac{p_k + n_k}{p + n} \quad \hat{n}_k = n \times \frac{p_k + n_k}{p + n} \quad \Delta = \sum_{k=1}^d \frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k}$$

# 决策树适用范围

- **缺失数据**：在许多问题领域中，并非每个样例的所有属性都是已知的。这些值可能没有被记录，也可能因获得它们的代价太大而无法获得。
- **连续属性与多值输入属性**：对于连续属性（如**身高**、**体重**或**时间**），可能每个样例都有不同的属性值。用信息增益来衡量属性将导致这样的属性得到理论上最高的信息增益，最终给出一棵以该属性为根的浅层树，其中每个可能值对应一个单样例子树。
- **连续值输出属性**：如果要预测一个数值类型的输出，那么我们需要的是**一棵回归树**（regression tree），而不是一棵分类树。回归树在每个叶节点上都有一个关于数值属性子集的线性函数，而不是一个单一的输出值。

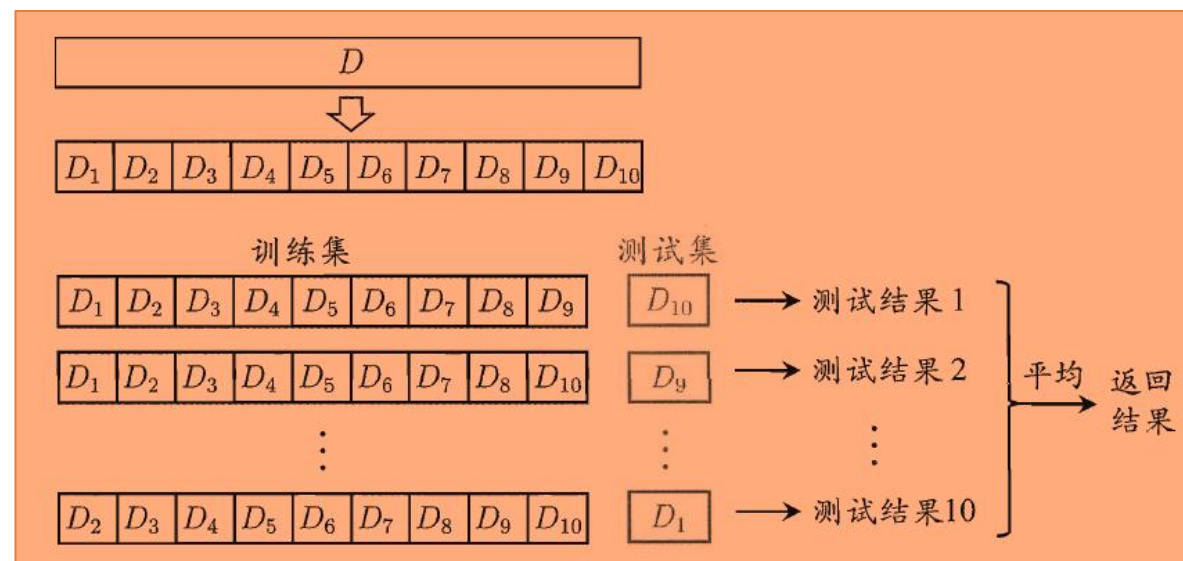
# 模型选择与模型优化

## 寻找一个好的假设的任务可以分为两个子任务

1. 模型选择 (Model selection) : 选择一个合适的假设空间
  2. 优化 (Optimization / Training) : 在该假设空间中找到最佳假设
    - 使用 训练集 (training set) 来创建假设
    - 使用 测试集 (test set) 来评估假设
- 平稳性 (stationary) 假设: 假设未来的样例类似于过去观测过的样本。我们假设每个样例  $E_j$  都具有相同的先验概率分布:  $P(E_j) = P(E_{j+1}) = P(E_{j+2}) = \dots$   
而且它与之前的样例是独立的:  $P(E_j) = P(E_j | E_{j-1}, E_{j-2}, \dots)$   
对于满足这些等式的样例, 我们称它们为独立同分布的或 i. i. d. 。
  - 错误率 (Error rate) : 在样本  $(x, y)$  上, 假设  $h(x) \neq y$  的比例

# 模型选择与模型优化

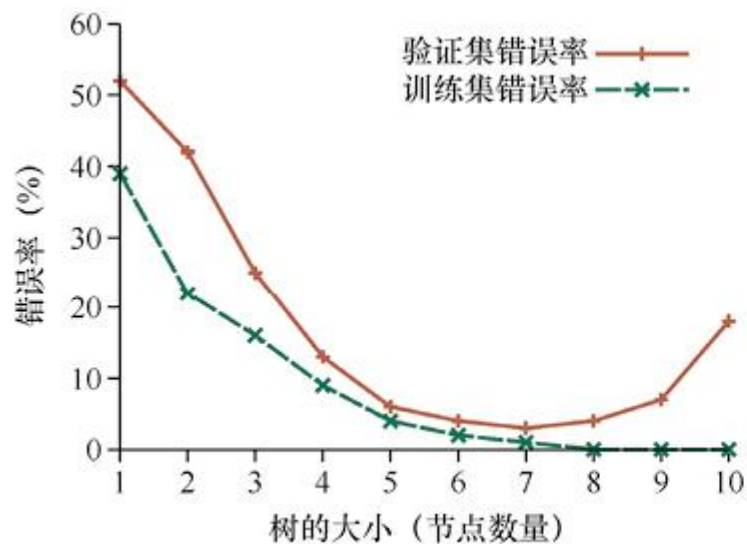
- **最佳拟合**：是最小化**错误率**（error rate）——对于样例(x, y),  $h(x) \neq y$ 的比例——的假设。可以通过对一个假设进行测试来估计其错误率：在一组称为**测试集**的样例上评估它的表现。
- 将拥有的样例分割成两组：一组为用于训练从而得到假设的**训练集**，另一组为用于评估假设的**测试集**。得到多个假设，通过在**验证集**（validation set）进行**超参数**（hyperparameter）调整，评估备选模型并选择最佳的备选模型。
- **k折交叉验证**（k-fold cross-validation）：每个样例都被作为训练数据和验证数据，从而提供双重功能，但又不同时提供。  
即使采用了交叉验证的方法，我们仍然需要一个单独的测试集。



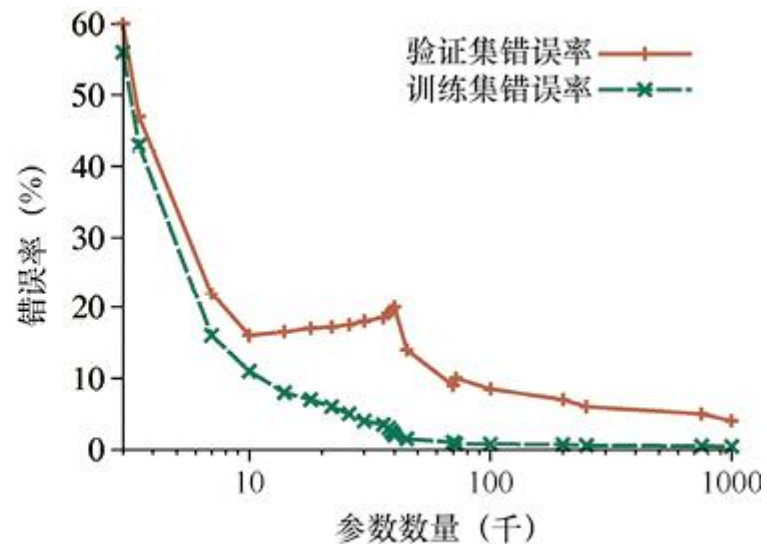


# 模型选择

模型选择中可能发生的两种典型的模式



(a)



(b)

两个不同问题上不同复杂性模型的训练误差（下方绿线）和验证误差（上方橙色线）。模型选择算法将选择验证误差最小的模型对应的超参数值。（a）模型类是决策树，超参数是节点数量。数据来自餐厅等待问题。最佳的超参数大小为7。（b）模型类是卷积神经网络，超参数是网络中常规参数的数量。数据是数字图像的MNIST数据集，任务是识别手写数字的照片。效果最好的超参数是1 000 000（注意坐标的对数刻度）



# 损失函数

通过最小化损失函数 (loss function) 最大化学习器的效用

$$L(x, y, \hat{y}) = \text{Utility}(\text{给定输入 } x, \text{ 使用 } y \text{ 的结果}) \\ - \text{Utility}(\text{给定输入 } x, \text{ 使用 } \hat{y} \text{ 的结果})$$

$$\text{绝对值损失: } L_1(y, \hat{y}) = |y - \hat{y}|$$

$$\text{平方误差损失: } L_2(y, \hat{y}) = (y - \hat{y})^2$$

$$\text{0/1损失: } L_{0/1}(y, \hat{y}) = 0 \text{ 若 } y = \hat{y}, \text{ 否则 } 1$$

经验损失 (empirical loss)

$$\text{EmpLoss}_{L,E}(h) = \sum_{(x,y) \in E} L(y, h(x)) \frac{1}{N}$$

得到的假设  $\hat{h}^*$  与真实函数  $f$  不同, 有4种可能的原因:

不可实现性、方差、噪声和计算复杂性

# 模型优化

**正则化** (regularization)，显式地惩罚复杂假设的复杂性：希望寻找更规则的函数。通过结合两种度量，即损失函数（L1或L2）和复杂性度量，称后者为**正则化函数** (regularization function)

$$Cost(h) = EmpLoss(h) + \lambda Complexity(h)$$

$$\hat{h}^* = \underset{h \in \mathcal{H}}{\operatorname{argmin}} Cost(h)$$

**特征选择** (feature selection)：判断属性的相关性，然后丢弃不相关的属性来减少模型的维数

## 超参数调整

- **手动调参** (hand-tuning)：根据个人以往的经验来猜测参数，在该参数下训练模型，并在验证集上测试其表现并分析结果，根据直觉得到参数调整的结果。
- **网格搜索** (grid search)：尝试所有超参数值的组合，观察哪个组合在验证集上表现得最好。
- **随机搜索** (random search) 采样
- **贝叶斯优化** (Bayesian optimization) 将选择好的超参数值的任务本身视为一个机器学习问题

# 学习理论

**概率近似正确** (*Probably approximately correct*, PAC): 在用少量的样例进行训练之后, 那些非常不匹配的假设将有很高的概率被“排除”, 因为它们将做出错误的预测。因此, 如果一个假设与足够多的训练样例相一致, 那么它不太可能是严重不匹配的

**PAC 学习算法**: 任何返回概率近似正确的假设的学习算法

假设  $h$  被称作近似正确的如果满足错误率  $\text{error}(h) \leq \epsilon$ ,  $\epsilon$  是一个较小的常数, 称  $h$  是**近似正确**的

$h_b$  与  $N$  个样例一致的概率上界:  $P(h_b \text{ agrees with } N \text{ examples}) \leq (1 - \epsilon)^N$ .

$H_b$  包含一个一致性假设的上界:  $P(\mathcal{H}_{\text{bad}} \text{ contains a consistent hypothesis}) \leq |\mathcal{H}_{\text{bad}}|(1 - \epsilon)^N \leq |\mathcal{H}|(1 - \epsilon)^N$

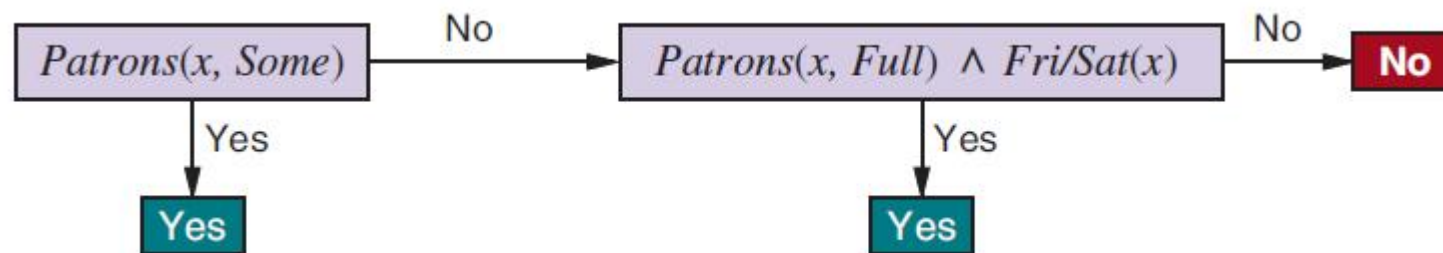
$$P(\mathcal{H}_{\text{bad}} \text{ contains a consistent hypothesis}) \leq |\mathcal{H}|(1 - \epsilon)^N \leq \delta.$$

$$N \geq \frac{1}{\epsilon} \left( \ln \frac{1}{\delta} + \ln |\mathcal{H}| \right) \quad \text{样本复杂性}$$

# 学习理论

## PAC学习示例：学习决策列表

$$WillWait \Leftrightarrow (Patrons = Some) \vee (Patrons = Full \wedge Fri/Sat)$$



$$|k\text{-DL}(n)| \leq 3^c c! \text{ where } c = |Conj(n, k)|.$$

$$|Conj(n, k)| = \sum_{i=0}^k \binom{2n}{i} = O(n^k).$$

$$|k\text{-DL}(n)| = 2^{O(n^k \log_2(n^k))}.$$

$$N \geq \frac{1}{\epsilon} \left( \ln \frac{1}{\delta} + O(n^k \log_2(n^k)) \right)$$

# 线性回归与分类

## 单变量线性回归

输入  $x$  和输出  $y$

$$y = w_1x + w_0$$

线性函数

$$h_{\mathbf{w}}(x) = w_1x + w_0$$

**线性回归:** 找到最匹配这些数据的线性函数  $h_{\mathbf{w}}$

需要做的就是找到对应的权重值  $(w_0, w_1)$  使得其经验损失最小.

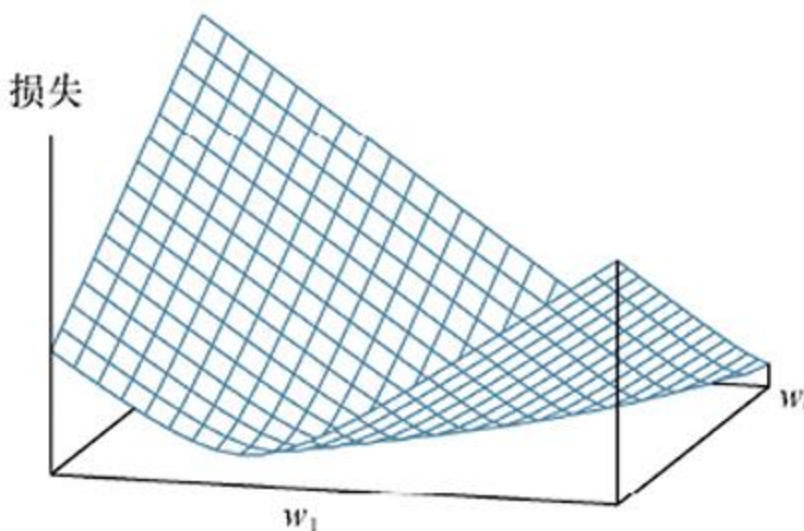
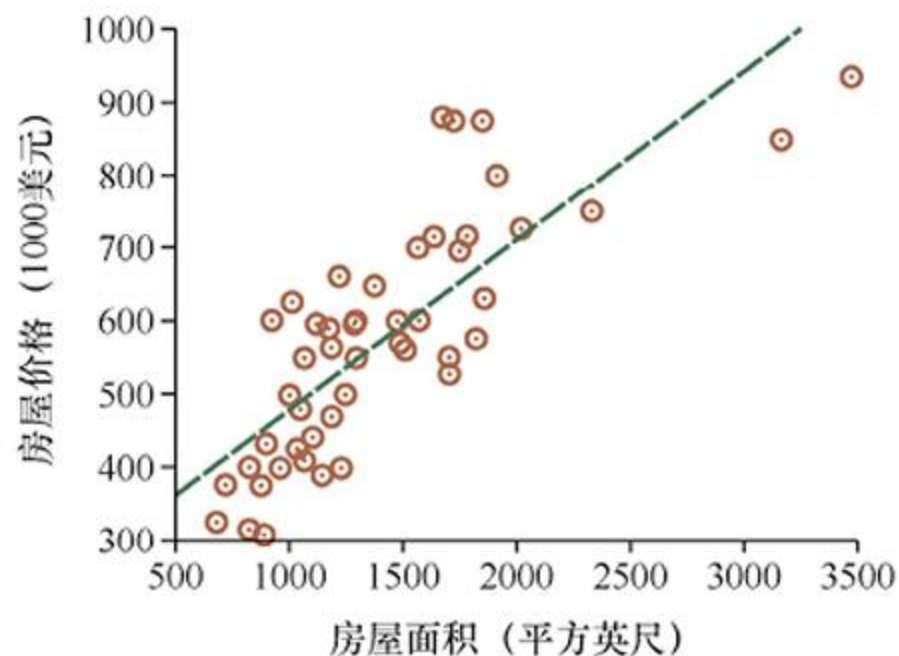
通常我们采用平方误差损失函数  $L_2$ , 并对所有训练样例进行求和

$$Loss(h_{\mathbf{w}}) = \sum_{j=1}^N L_2(y_j, h_{\mathbf{w}}(x_j)) = \sum_{j=1}^N (y_j - h_{\mathbf{w}}(x_j))^2 = \sum_{j=1}^N (y_j - (w_1x_j + w_0))^2.$$

$$\frac{\partial}{\partial w_0} \sum_{j=1}^N (y_j - (w_1x_j + w_0))^2 = 0 \text{ and } \frac{\partial}{\partial w_1} \sum_{j=1}^N (y_j - (w_1x_j + w_0))^2 = 0.$$

此时该问题有唯一解:  $w_1 = \frac{N(\sum x_j y_j) - (\sum x_j)(\sum y_j)}{N(\sum x_j^2) - (\sum x_j)^2}$ ;  $w_0 = (\sum y_j - w_1(\sum x_j))/N$ .

# 线性回归与分类



- (a) 2009年7月加州伯克利地区房屋售价与面积关系的数据点，以及最小化平方误差损失的线性函数假设： $y = 0.232x + 246$ 。
- (b) 损失函数  $\sum_j (y_j - w_1 x_j + w_0)^2$  随参数  $w_0$  和  $w_1$  变化的曲线图。注意该损失函数是凸函数，具有唯一全局最小值。



# 线性回归与分类

## 梯度下降

- 通过逐步修改参数来搜索连续的权重空间使得损失最小化

```
w ← any point in the parameter space  
while not converged do  
  for each  $w_i$  in w do  
     $w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w})$ 
```

- $\alpha$ : 步长/学习率，可以是一个固定的常数，也可以随着学习过程的进行逐渐衰减

$$\begin{aligned}\frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w}) &= \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(x))^2 = 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i} (y - h_{\mathbf{w}}(x)) \\ &= 2(y - h_{\mathbf{w}}(x)) \times \frac{\partial}{\partial w_i} (y - (w_1 x + w_0)).\end{aligned}$$

$$\frac{\partial}{\partial w_0} \text{Loss}(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \quad \frac{\partial}{\partial w_1} \text{Loss}(\mathbf{w}) = -2(y - h_{\mathbf{w}}(x)) \times x.$$

# 线性回归与分类

## 批梯度下降 *Batch gradient descent*

最小化每个样例各自损失的总和

$$w_0 \leftarrow w_0 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)); \quad w_1 \leftarrow w_1 + \alpha \sum_j (y_j - h_{\mathbf{w}}(x_j)) \times x_j.$$

- 其损失曲面是凸的,
- 着训练不会陷入局部极小值, 到全局最小值的收敛性可以保证
- 只要我们不选择一个过大的 $\alpha$
- **Epoch**: 遍历了所有训练样例的一步更新

## 随机梯度下降 *Stochastic gradient descent or SGD*

- 在每一步中随机选择少量训练样例
- 从  $N$  个样例中选择一个大小为  $m$  的小批量.
- 选择合适的  $m$  来利用并行向量运算
- 在线梯度下降



# 线性回归与分类

## 多变量线性回归

- $\mathbf{x}_j$  是  $n$  元向量
- $h$ : 用权重与 输入向量的点积

$$h_{\mathbf{w}}(\mathbf{x}_j) = \mathbf{w} \cdot \mathbf{x}_j = \mathbf{w}^\top \mathbf{x}_j = \sum_i w_i x_{j,i}$$

- 最佳的权重向量,  $\mathbf{w}^*$ : 为最小化样例上的平方误差损失

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w}} \sum_j L_2(y_j, \mathbf{w} \cdot \mathbf{x}_j).$$

- 梯度下降将收敛到损失函数的（唯一）最小值，每个权重  $w_i$  的更新式为

$$w_i \leftarrow w_i + \alpha \sum_j (y_j - h_{\mathbf{w}}(\mathbf{x}_j)) \times x_{j,i}.$$

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \quad \text{正规方程}$$

# 线性回归与分类

## 多变量线性回归

- 多变量线性回归容易导致过拟合
- 正则化

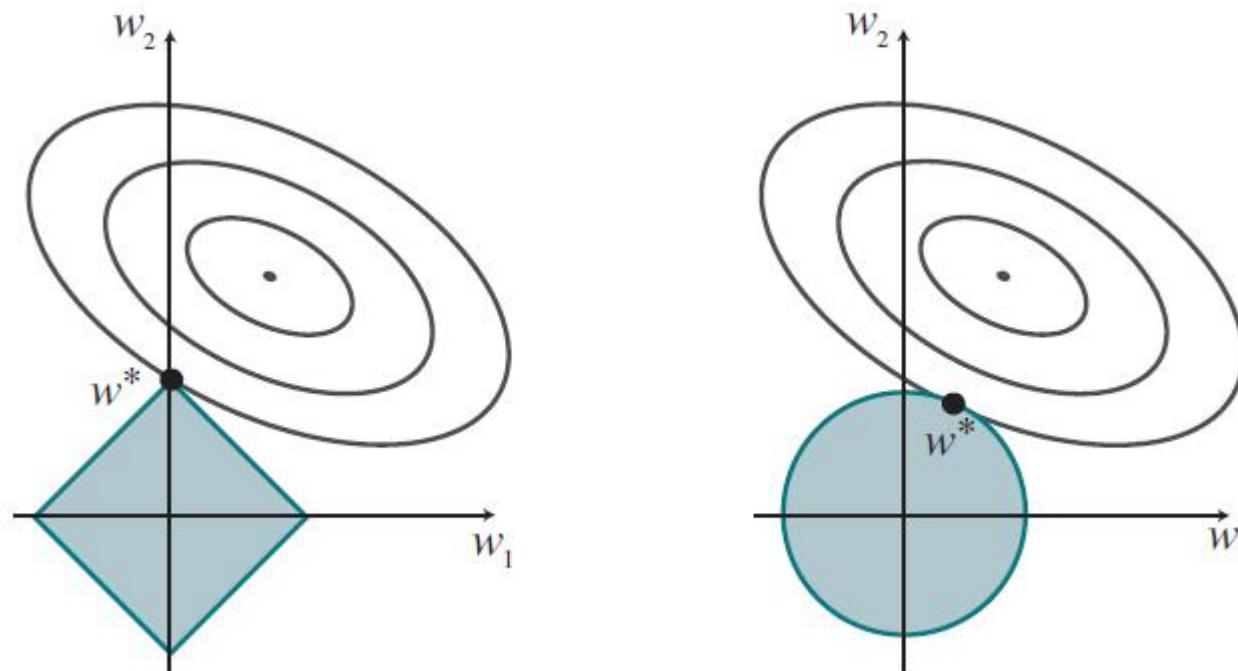
$$Cost(h) = EmpLoss(h) + \lambda Complexity(h)$$

- 函数的复杂性取决于权重的大小，考虑如下的正则化函数族

$$Complexity(h_{\mathbf{w}}) = L_q(\mathbf{w}) = \sum_i |w_i|^q$$

- $q=1$ : L1正则化  $q=2$ : L2正则化

# 线性回归与分类



$L_1$  正则化倾向于产生一个稀疏模型的解释. 左: 在  $L_1$  正则化 (菱形 框) 的情况下, 正则化约束内的最小损失 (同心等高线) 一般会出现在某一条轴 上, 这意味着一个权重为0; 右: 在  $L_2$  正则化 (圆形) 的情况下, 最小损失可能出 现在圆周上的任何位置, 因而不会偏向0权重

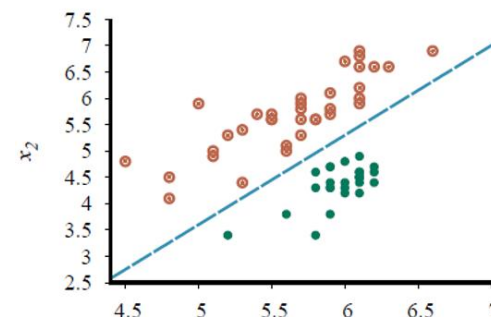
# 线性回归与分类

## 带有硬阈值的线性分类器

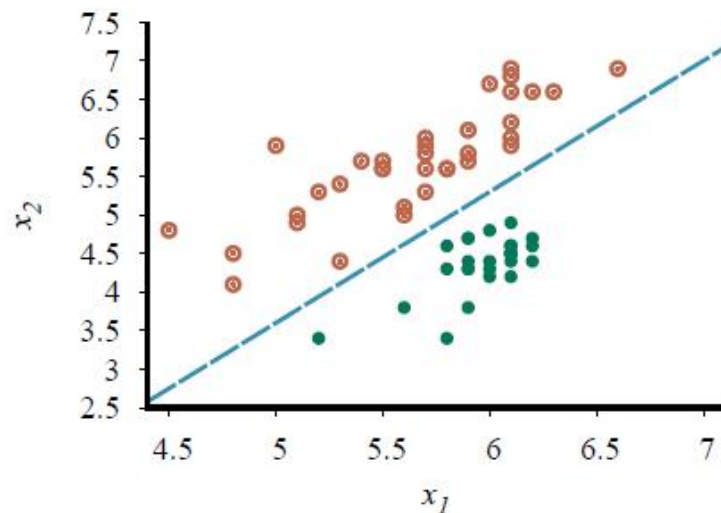
- 线性函数可用于回归，也可以用于分类。
- 决策边界是一条直线、一个平面或者更高维的面，它将两类数据分离。
- 线性分离器：线性决策边界，允许这种分离器存在的数据我们称之为线性可分的数据
- $h$ : 以线性函数  $\mathbf{w} \cdot \mathbf{x}$  作为输入，通过定义一个阈值函数来表示的假设

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Threshold}(\mathbf{w} \cdot \mathbf{x}) \text{ where } \text{Threshold}(z) = 1 \text{ if } z \geq 0 \text{ and } 0 \text{ otherwise.}$$

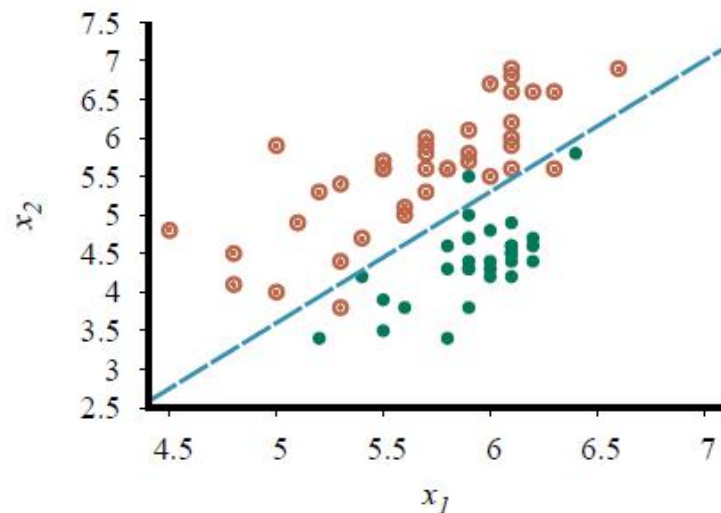
- 感知机学习规则：
$$w_i \leftarrow w_i + \alpha(y - h_{\mathbf{w}}(\mathbf{x})) \times x_i$$
- 3种可能发生的状况：
  - 若输出是正确的  $y = h_{\mathbf{w}}(\mathbf{x})$ ：那么权重将不会改变
  - 如果  $y$  为1而  $h_{\mathbf{w}}(\mathbf{x})$  为0，那么当相应的输入  $x_i$  是正数时  $w_i$  将增大，当  $x_i$  是负数时  $w_i$  将减小。这是有直观意义的，因为我们希望相应的会更大，进而使得  $h_{\mathbf{w}}(\mathbf{x})$  的输出为1.
  - 如果  $y$  为0而  $h_{\mathbf{w}}(\mathbf{x})$  为1，那么当相应的输入  $x_i$  是正数时  $w_i$  将减小，当  $x_i$  是负数时  $w_i$  将增大。这是有直观意义的，因为我们希望相应的会更小，进而使得  $h_{\mathbf{w}}(\mathbf{x})$  的输出为0.



# 线性回归与分类



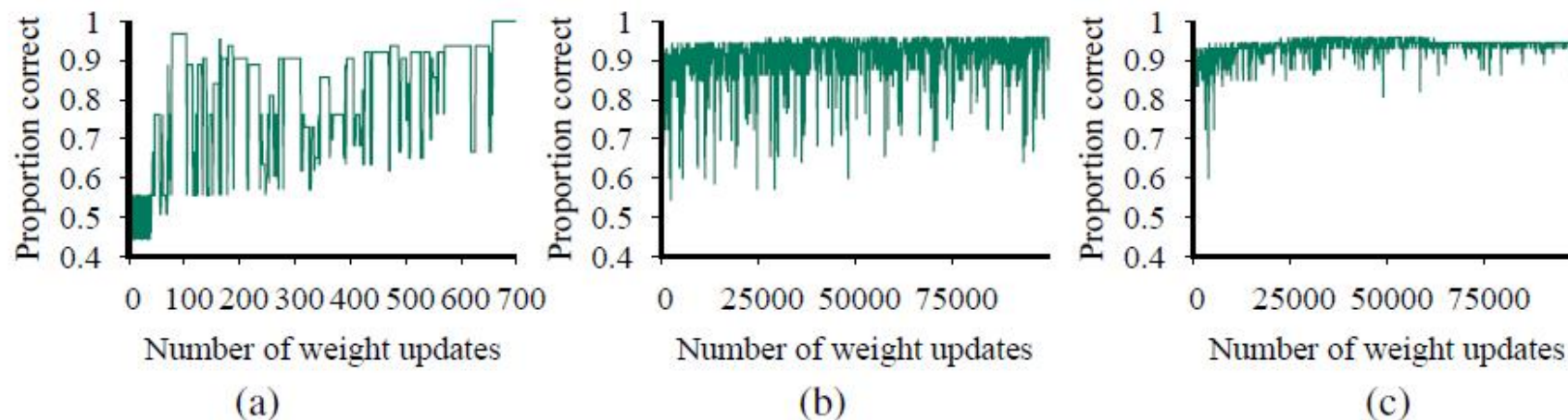
(a)



(b)

- (a) 两种类型地震数据，包含体波震级  $x_1$  和面波震级  $x_2$ ，数据来源于 1982~1990 年在亚洲和中东发生的地震（橙色空心圆）和地底爆炸（绿色实心圆）。图中还绘制了类之间的决策边界。
- (b) 同一个领域，有比之前更多的数据点。此时地震和爆炸不再是线性可分的

# 线性回归与分类



- (a) 给定地震/爆炸数据，模型在总训练集上的准确度与感知机学习规则训练的迭代次数的关系图。
- (b) 对图中带噪声的、不可分离的数据绘制相同的图；注意 x 轴刻度有所改变。
- (c) 学习率为  $a(t) = 1000/(1000 + t)$  时，绘制与 (b) 中相同的图。

# 线性回归与分类

## 基于逻辑斯谛回归的线性分类器

- 软阈值 — 通过使用一个连续的、可微的函数来近似硬阈值函数

- 逻辑斯谛函数:

$$\text{Logistic}(z) = \frac{1}{1 + e^{-z}}$$

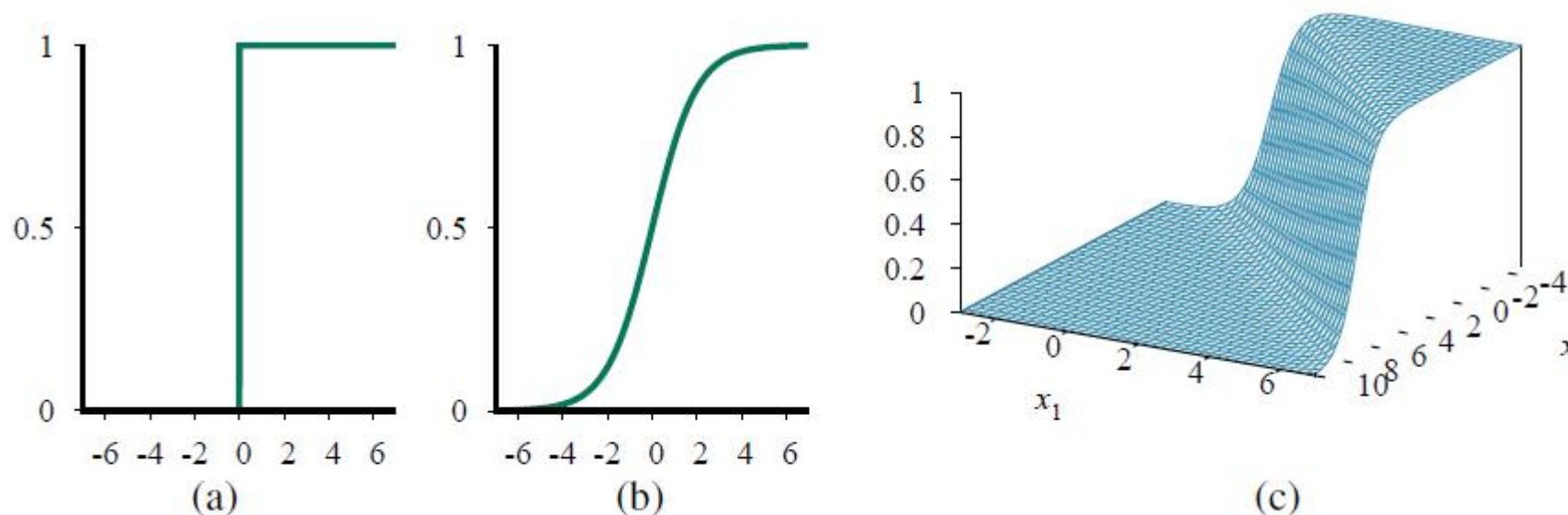
- 逻辑斯谛函数代替阈值函数，可以得到：

$$h_{\mathbf{w}}(\mathbf{x}) = \text{Logistic}(\mathbf{w} \cdot \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w} \cdot \mathbf{x}}}.$$

- 逻辑斯谛回归: 拟合模型的权重以最小化数据集上的损失的过程



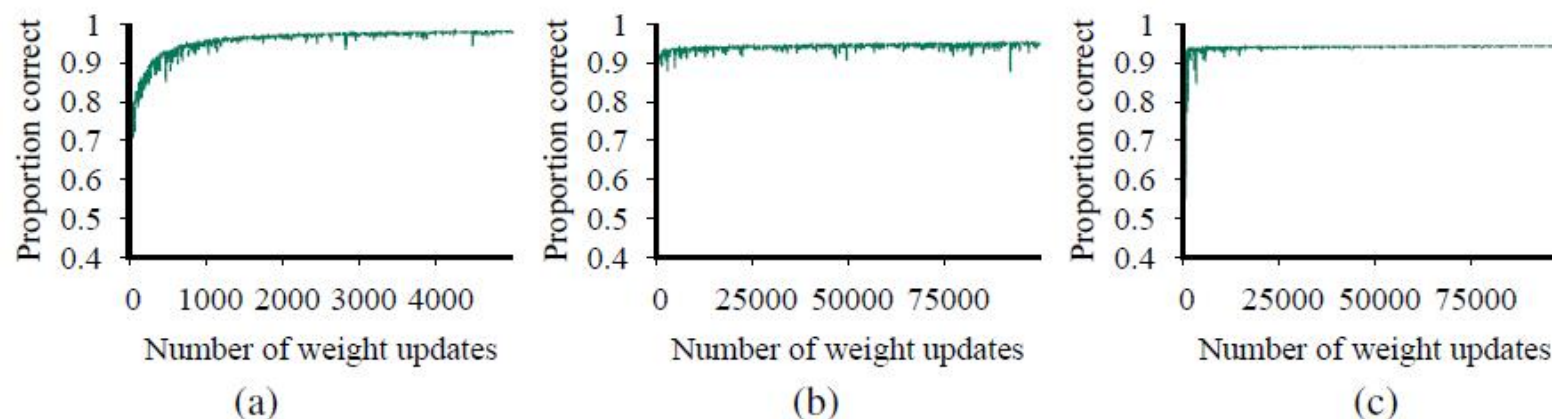
# 线性回归与分类



- (a) 硬阈值函数  $Threshold(z)$  输出为 0/1. 注意, 该函数在  $z = 0$  处是不可微的.
- (b) 逻辑斯谛函数  $Logistic(z) = \frac{1}{1+e^{-z}}$  也称为sigmoid函数
- (c) 对于地震/爆炸问题数据的逻辑斯谛回归假设  $h_w(\mathbf{x}) = Logistic(\mathbf{w} \cdot \mathbf{x})$

1

# 线性回归与分类



重复使用逻辑斯谛回归进行之前的实验. (a) 中的图包含了5000 次迭代, 而不是700次, 而 (b) 和 (c) 中的图使用与之前相同的坐标刻度

# 非参数模型

**参数模型:** 通过参数个数固定的参数集（与训练样例的数量无关）来汇总数据信息的学习模型

**非参数模型:** 无法用参数个数固定的参数集来表示的模型

例如，分段线性函数将所有数据点 保留为模型的一部分。这样做的学习方法也被称为 基于实例的学习（instance-based learning）或基于记忆的学习（memory-based learning）

最简单的基于实例的学习方法是**查表**（table lookup）：  
取所有的训练样例，将它们放在查找表中，然后在需要访问  $h(x)$  时，查看  $x$  是否在表中；  
如果是，则返回相应的  $y$

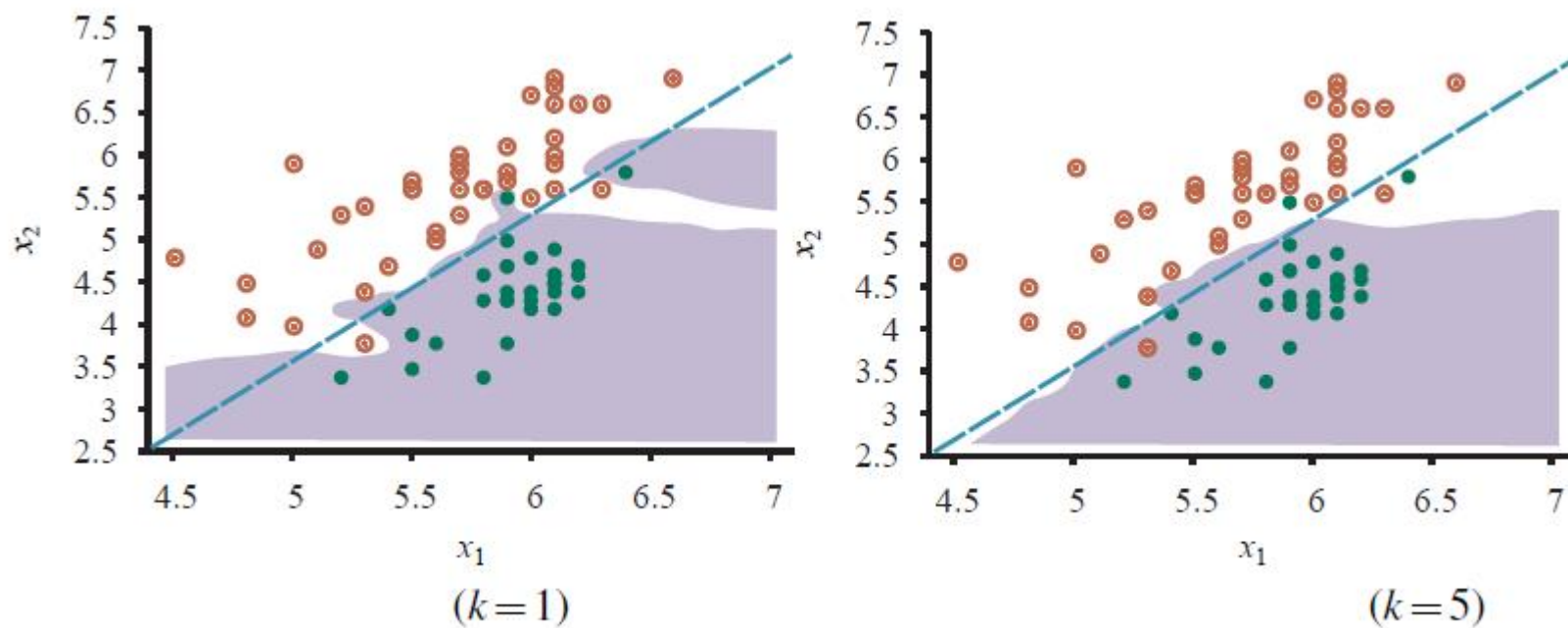
# 非参数模型

## 最近邻模型

### **$k$ -nearest-neighbors**

- 给定待查询的 $\mathbf{x}_q$ , 寻找最接近它的 $k$  个样例, 而不是寻找等于  $\mathbf{x}_q$  的样例
- $\mathbf{x}_q$  的 $k$  个最近邻邻居的集合  $= NN(k, \mathbf{x}_q)$
- 例如, 如果  $k = 3$  且输出值为 *Yes, No, Yes* , 则分类结果将为 *Yes*.
- 为了避免二分类结果数量相同, 通常将,  $k$  选择为奇数.

# 非参数模型



- (a) 一个  $k$  近邻模型，该图显示了地震数据的爆炸一类的范围，其中  $k = 1$ 。可以明显看到有过拟合现象。
- (b)  $k = 5$  时，该数据集上的过拟合现象消失了。

# 非参数模型

## 最近邻模型

### **k-nearest-neighbors**

- 测量从查询点  $\mathbf{x}_q$  到样例点  $\mathbf{x}_j$  的距离，通常使用 闵可夫斯基距离 **Minkowski distance** 或  $L^p$  范数：

$$L^p(\mathbf{x}_j, \mathbf{x}_q) = \left( \sum_i |x_{j,i} - x_{q,i}|^p \right)^{1/p}.$$

- $p=2$ , 欧几里得距离
- $p=1$ , 曼哈顿距离
- 对于布尔属性值，两个数据点不同的属性的数量称为 汉明距离（Hamming distance）
- 马氏距离 **Mahalanobis distance**: 考虑了维度之间的协方差.
- 维度灾难Curse of Dimensionality）：是指随着数据维度（特征数量）增加，许多常用的机器学习方法、统计分析方法会变得计算困难、数据稀疏、泛化性能下降的一系列现象的统称。

# 非参数模型

## 局部敏感哈希

- 哈希表是一个比二叉树更快捷的查找方式
- 距离近的数据点分在同一组并分布于同一个箱子中，因此我们需要使用 局部敏感哈希 (Locality Sensitive Hash, *LSH*)
- 创建了多个随机投影并将其进行组合
  - 随机 投影只是位串表示的一个随机子集
  - 选择  $e$  个不同的随机投影并创建  $e$  个哈希表  $g_1(\mathbf{x}), \dots, g_e(\mathbf{x})$ .
  - 将所有样例输入到每个哈希表中
  - 给定查询点  $\mathbf{x}_q$  在每个哈希表箱子  $g_i(\mathbf{x}_q)$  中获取点集,
  - 将这些集合取并集作为一组候选点  $C$ .
  - 计算  $\mathbf{x}_q$  到  $C$  中每一个点的实际距离并返回其中  $k$  个距离最近的点



# 非参数模型

## 非参数回归

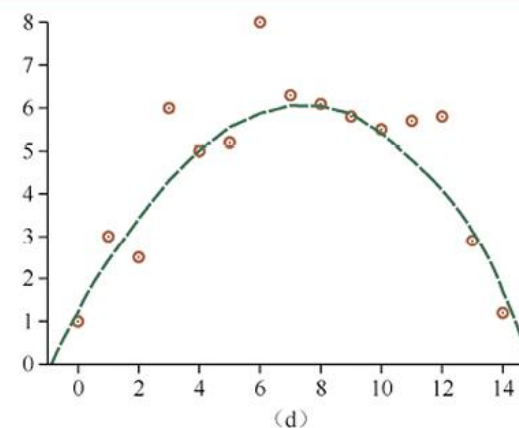
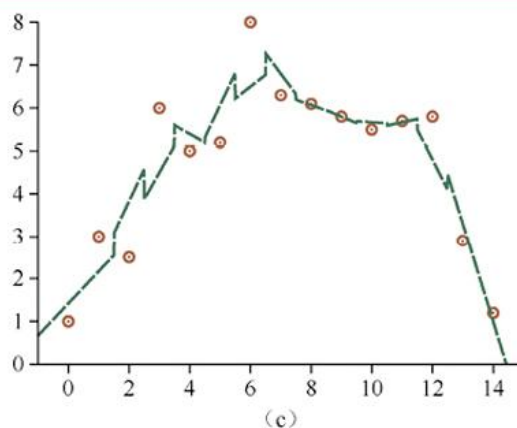
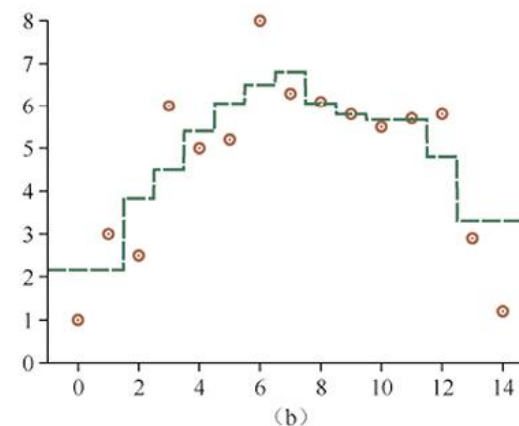
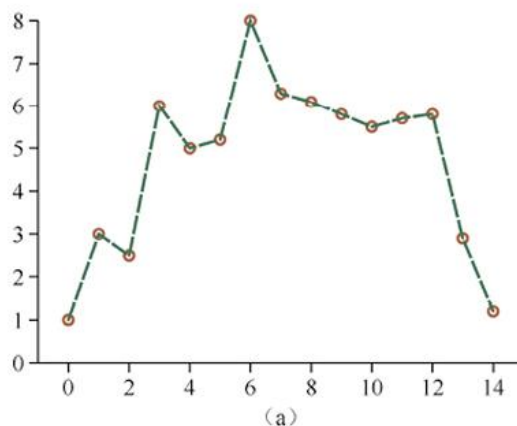
- 点连接
- $k$ -近邻回归在点连接的基础上做了改善，不使用查询点  $\mathbf{x}_q$  左右两侧的两个样例进行预测，而是使用  $k$ 个最近邻。
- 局部加权回归: 利用了近邻的优势，且没有不连续性
  - 在每个查询点  $\mathbf{x}_q$ , 接近  $\mathbf{x}_q$  的样例将被赋予较高的权重，而距离较远的样例权重较小，最远的样例则将没有权重。权重的大小随距离的变化通常是渐变的，而不是突变的。
  - 使用一个 **核**（kernel）来确定每个样例的权重，该函数的输入是查询点与样例点之间的距离。对于给定的查询点  $\mathbf{x}_q$  只需要解决以下加权回归问题：

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} \sum_j \mathcal{K}(\operatorname{Distance}(\mathbf{x}_q, \mathbf{x}_j)) (y_j - \mathbf{w} \cdot \mathbf{x}_j)^2,$$

$$h(\mathbf{x}_q) = \mathbf{w}^* \cdot \mathbf{x}_q$$

# 非参数模型

## 非参数回归



非参数回归模型：(a) 直接连接数据点。(b) 3近邻均值模型。(c) 3近邻线性回归模型。(d) 局部加权回归，核宽度为10

# 非参数模型

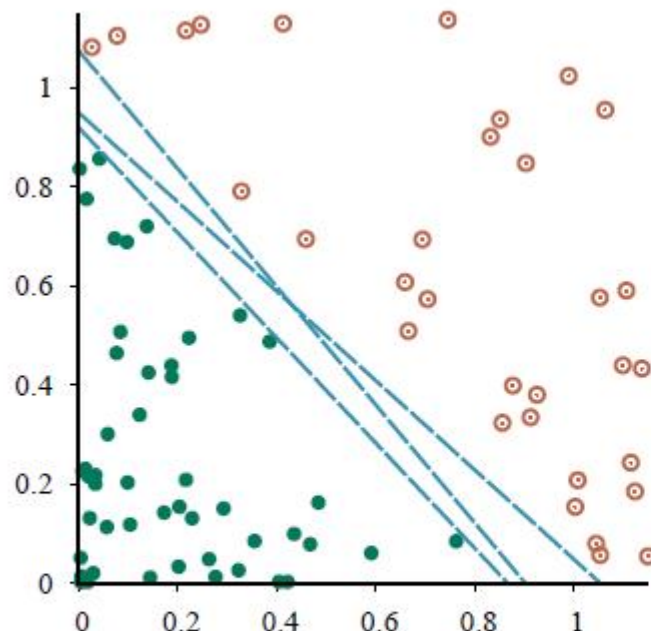
## 支持向量机(SVM)

最流行的解决监督学习问题的“现成”方法:

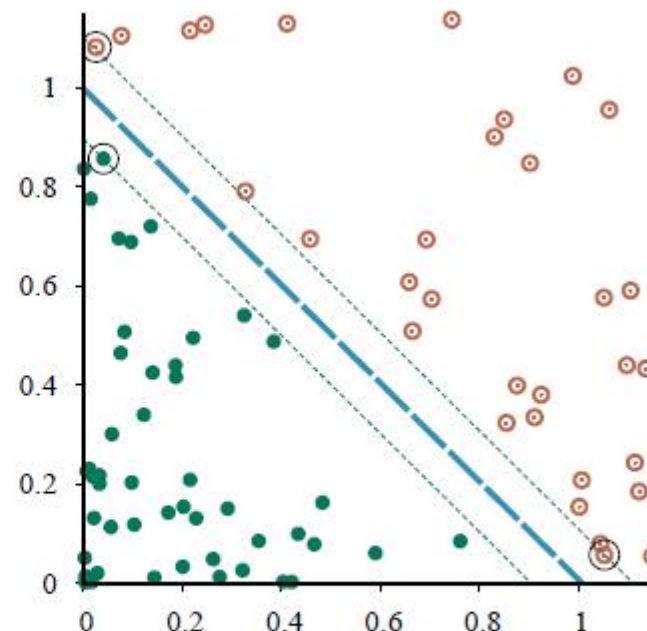
- 支持向量机构造了一个**极大边距分离器**，即到样例点距离最大的决策边界。这有助于它们较好地泛化。
- 支持向量机构造了一个**线性分离超平面**，但是它们能通过使用所谓的**核技巧**来将数据嵌入更高维空间中。通常原始输入空间中不是线性可分的数据很容易在高维空间中被分离。
- 支持向量机是**非参数**的——其分离超平面是由一组样例点而不是参数值的集合定义的。虽然例如最近邻等模型需要保留所有样例的信息，但SVM模型仅需要保留最靠近分离平面的样例——通常是一个较小的常数乘以维数。因此，SVM结合了非参数模型和参数模型的优点：它们可以灵活地表示复杂的函数，同时可以防止过拟合。

**SVM并不是最小化训练数据上的期望经验损失，而是最小化期望泛化损失。**

# 非参数模型



(a)



(b)

支持向量机分类器：

(a) 两类点（橙色空心圆和绿色实心圆）和3个候选线性分离器。

(b) 极大边距分离器（粗线）位于边距（虚线之间的区域）的中间。支持向量（带有大黑圈的点）是最靠近分离器的，本图中有3个

# 非参数模型

## 支持向量机(SVM)

- 分离器定义为点集  $\{\mathbf{x} : \mathbf{w} \cdot \mathbf{x} + b = 0\}$ .
- 通过求解对偶问题, 可找到原问题的最优解

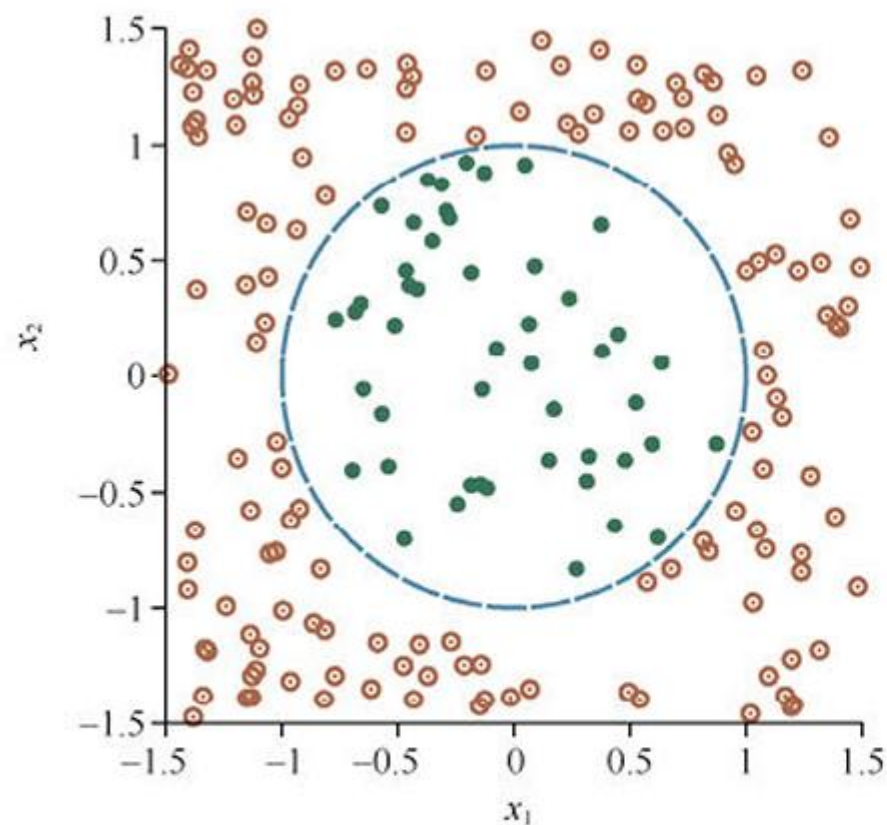
$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k) \quad \alpha_j \geq 0 \quad \sum_j \alpha_j y_j = 0$$

- 其表达式是凸的; 因此它有一个可以被有效方法找到的全局最大值.
- 数据仅以成对数据的点积形式输入表达式
- 分离器为:

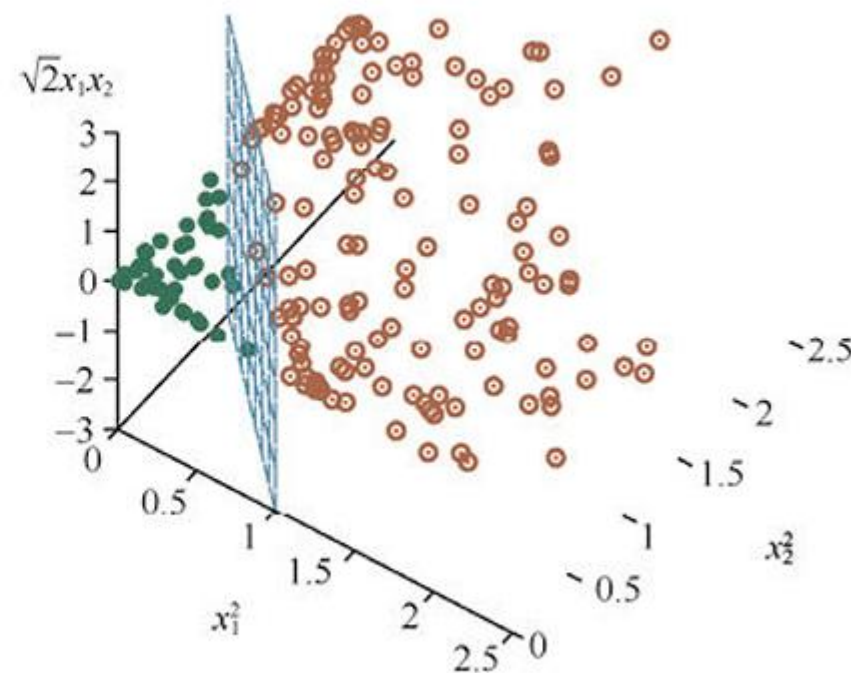
$$h(\mathbf{x}) = \operatorname{sign} \left( \sum_j \alpha_j y_j (\mathbf{x} \cdot \mathbf{x}_j) - b \right)$$

- 最后一个重要的性质是, 除支持向量 (support vector, 即最靠近分离器的点) 外, 每个数据点对应的权重  $\alpha_j$  为 0。

# 非参数模型



(a)



(b)

(a) 一个二维训练集，其中正样例为绿色实心圆，负样例为橙色空心圆。图中还给出了真实的决策边界

(b) 映射到三维输入空间后的相同数据。(a) 中的圆形决策边界变成 三维空间中的线性决策边界



# 非参数模型

## 核技巧

- mercer定理：任何“合理的”（即生成矩阵是正定的）核函数，都对应于某个特征空间
- 将核函数代入下式，可以在数十亿维度（甚至无限多维）的特征空间中高效地找到最优线性分离器

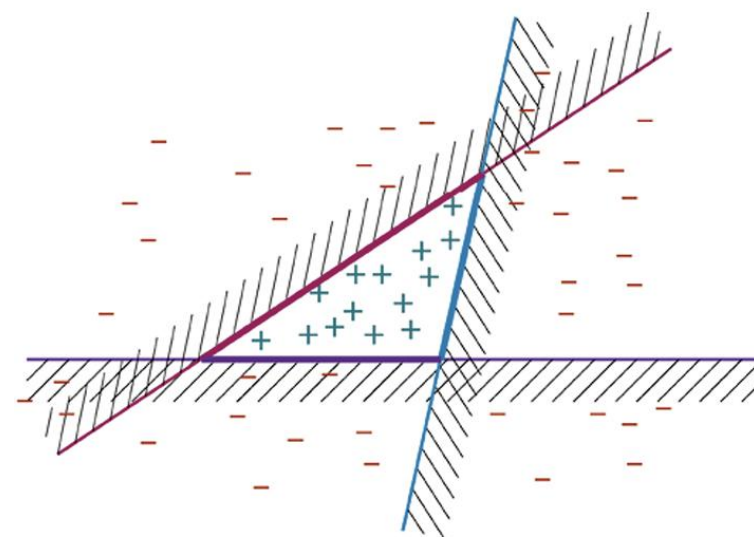
$$\operatorname{argmax}_{\alpha} \sum_j \alpha_j - \frac{1}{2} \sum_{j,k} \alpha_j \alpha_k y_j y_k (\mathbf{x}_j \cdot \mathbf{x}_k)$$

- 将生成的线性分离器映射回原始输入空间后，可以对应于任意一个在正样例和负样例之间的波动的非线性决策边界。
- 核方法不仅可以用于寻找最优线性分离器的学习算法，而且可以应用于任何其他可描述为仅用到数据对点积的算法，在将算法描述为点积后，将点积替换为核函数，我们就有了该算法的核化（kernelization）算法。



# 集成学习

- 集成学习（ensemble learning）选择一个由一系列假设,  $h_1, h_2, \dots, h_n$  构成的集合, 通过平均、投票或其他形式的机器学习方法将它们 的预测进行组合。
- 单独的假设为 **基模型**（base model）
- 组合后的模型为 **集成模型**（ensemble model）
- 选择集成学习的原因
  - 减少偏差。基模型的假 设空间可能过于局限, 进而造成了很大的偏差（例如逻辑斯谛回归中线性决策边界的偏差）。与基模型相比, 集成模型有更强大的表达能力, 因此偏差会较小。
  - 减少方差, 如果集成模型误分类了一个新的样例, 那么 $k$ 个基分类器中至少有 $k/2+1$ 个对样例进行了误分类。我们希望这个概率比单个分类器进行错误分类的概率要小。



通过集成学习的方法提升表达能力。我们使用3个线性阈值假设, 每个假设在不带阴影的一面都是正分类, 将所有被这3个面同时分类为正的样例分类为正。所得的三角形区域是在原始假设空间中无法被表达的假设

# 集成学习

**例：**假设已经训练了一个在80%的情况下能分类正确的分类器。现在构造一个由5个分类器集成的模型，其中每个基分类器都在数据集的不同子集上进行训练，因而它们将是独立的。假设这么做会导致单个训练器性能下降，每个分类器仅在75%的情况下是正确的，则5个独立分类器多数表决的正确率为：

假设有  $n$  个互相独立且每个都以概率  $p$  正确的分类器。采用多数表决 (majority vote) 时，整体正确的概率等于：至少有超过半数分类器正确的概率，即

$$P(\text{majority correct}) = \sum_{k=\lceil \frac{n}{2} \rceil}^n \binom{n}{k} p^k (1-p)^{n-k}.$$

这是二项分布上半尾的和（每一项表示恰有  $k$  个分类器正确的概率）。

**$n=5$** ，每个分类器  **$p=0.75$** ，多数需要至少 3 个正确 ( $k=3,4,5$ )，所以

$$P = \sum_{k=3}^5 \binom{5}{k} (0.75)^k (0.25)^{5-k}.$$

逐项相加后可得多数表决正确的概率为：0.896484375 $\approx$ 89.65%

$n=17$ 时， $P \approx 0.9876152206 \approx 98.76\%$

# 集成学习

## 自助聚合法(Bagging)

- 通过采样并替换原始训练集来生成  $K$  个不同的训练集.
- 从训练集中随机选取  $N$  个 样例, 这些样本中的每一个都可能是我们之前选择过的一个样例
- 在这  $N$  个样例上运行机器学习算法, 并获得一个假设
- 重复这个过程  $K$  次即得到  $K$  个不同的假设
- 汇总来自所有  $K$  个假设的预测.
- 对于分类问题, 这意味着 进行相对多数投票 (对于二分类, 为绝对多数投票)。
- 对于回归问题, 最终输出为平均值:

$$h(\mathbf{x}) = \frac{1}{K} \sum_{i=1}^K h_i(\mathbf{x})$$

减少方差

# 集成学习

## 随机森林Random forests

- 决策树自助聚合法的一种形式
- 主要思想: 随机化属性选择
- 在构造树的过程中, 对于每个分割点, 我们从属性中随机抽样, 并计算哪个属性有最高的信息增益。
- 如果共有  $n$  个属性, 常用的默认选择是在每个分割点随机选择  $\sqrt{n}$  个属性 (对于分类问题), 或选择  $n/3$  个属性 (对于回归问题)。
- 极端随机树(*Extremely randomized trees* ,*ExtraTrees*): 在选择分割点数值时引入随机性
  - 对于 每个选定的属性, 我们从属性值域内随机均匀地采样几个候选值;
  - 选择其中具有最高信息增益的值。
- 随机森林是一个复杂的、未剪枝的模型, 但是它们可以避免过拟合  
随着树数量增加, 随机森林通过投票降低方差, 使误差趋于收敛、不再增大, 但由于偏差仍然存在, 因此并不能完全避免过拟合, 也不可能使误差趋于零

# 集成学习

## 堆叠法(Stacking)

- 在相同数据上使用不同模型类训练的多个基模型进行组合
- 方法
  - 使用相同训练集来训练不同的基模型,
  - 使用留出的 验证数据 (加上基模型预测) 来训练集成模型.
  - 如果需要, 也可以使用交叉验证.
- 该方法称为“堆叠”, 因为可以将其视为由基模型构成第一层, 并在基模型层的输出上进行操作进而建立的集成模型。实际上, 我们也可以堆叠多个层, 每一层都在前一层的输出上进行操作。堆叠的方法可以减小偏差, 并且通常所获得的模型性能要优于任何单个基模型。

训练集  $\mathbf{x}_1 = \text{Yes, No, No, Yes, Some, $$$, No, Yes, French, } 0 \sim 10; y_1 = \text{Yes}$

不同基模型

验证集  $\mathbf{x}_2 = \text{Yes, No, No, Yes, Full, \$, No, No, Thai, } 30 \sim 60, \text{Yes, No, No}; y_2 = \text{No}$   
(原始数据+合适的预测)

集成模型

# 集成学习

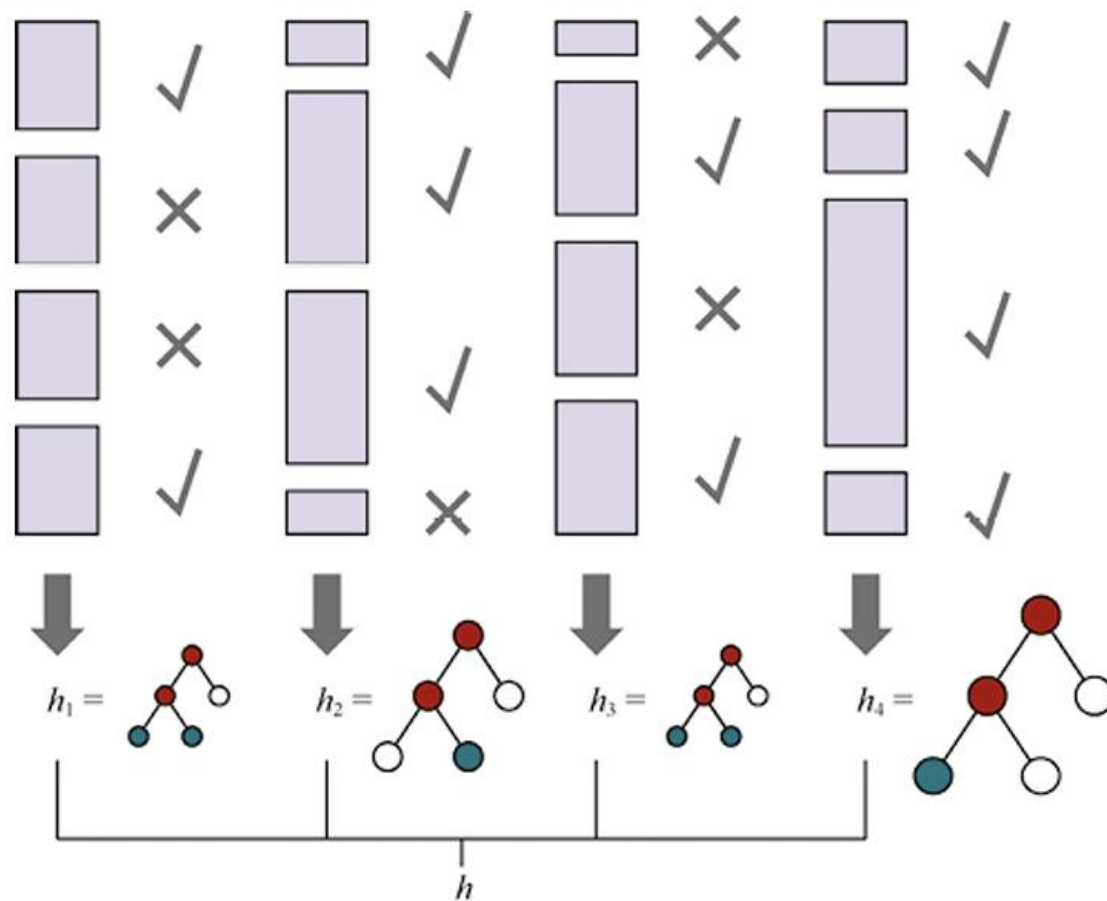
## 自适应提升法(Boosting)

- 加权训练集: 给每个样例赋予一个权重  $w_j \geq 0$  该权重描述了样例在训练过程中应计数的次数。
- 从所有样例具有相等的权重  $w_j = 1$  开始。根据该训练集, 我们训练第一个假设  $h_1$ .
- 希望下一个假设能在被分类错误的样例中表现得更好, 因此将增加它们的权重, 同时减小正确分类的样例的权重。基于这个重新进行加权得到的训练集, 我们训练得到新的假设。
- 重复进行以上过程, 直到生成  $K$  个假设, 其中  $K$  是自适应提升算法的输入之一。
- 值得注意的是, 这是一个贪心算法, 且它不会回退, 即一旦算法选择了某个假设  $h_i$  它就永远不会抛弃该选择, 而是会添加新的假设。它也是一种序贯算法, 因此我们无法像自助聚合法一样并行计算所有假设。

$$h(\mathbf{x}) = \sum_{i=1}^K z_i h_i(\mathbf{x})$$



# 集成学习



自适应提升算法的运作方式。每个阴影矩形对应一个样例。矩形的高度对应于样例的权重。勾和叉号表示该样例是否在当前假设下正确分类。决策树的大小代表该假设在最终集成模型中的权重大小



# 集成学习

## 输入的学习算法( $L$ )

**弱学习算法:**  $L$  总是在训练集上返回准确度略高于随机猜想的假设（即，对布尔分类问题来说，假设的准确度为  $50\% + \epsilon$ ）

## ADABOOST

- 如果输入的学习算法  $L$  是 弱学习 算法将返回一个假设，对于足够大的  $K$ ，该假设能对训练数据进行完美的分类
- 算法提升了原始学习算法对训练数据预测的准确性
- 只要基模型比随机猜测更好，我们就可以通过自适应提升的方法消除基模型中的任何大小的偏差，

# 集成学习

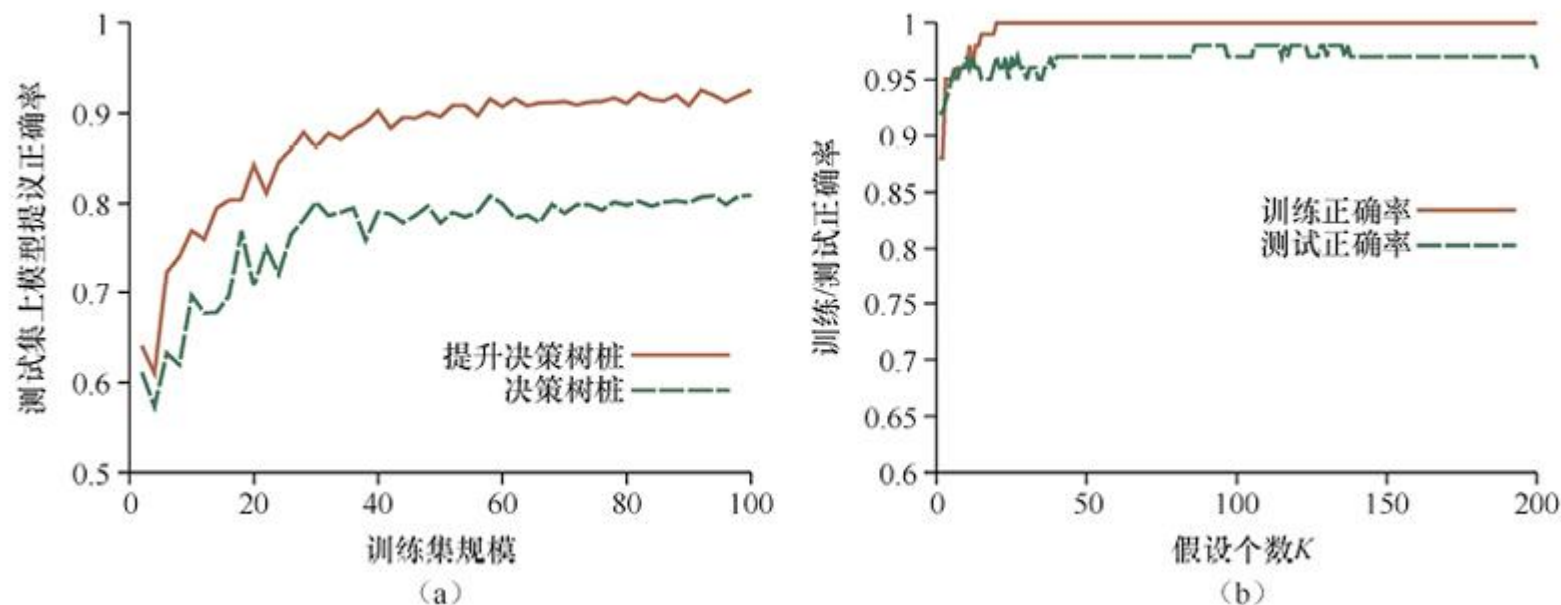
```

function ADABOOST(examples, L, K) returns a hypothesis
  inputs: examples, set of N labeled examples  $(x_1, y_1), \dots, (x_N, y_N)$ 
           L, a learning algorithm
           K, the number of hypotheses in the ensemble
  local variables: w, a vector of N example weights, initially all  $1/N$ 
                     h, a vector of K hypotheses
                     z, a vector of K hypothesis weights

   $\epsilon \leftarrow$  a small positive number, used to avoid division by zero
  for k = 1 to K do
    h[k]  $\leftarrow L(\text{examples}, \mathbf{w})$ 
    error  $\leftarrow 0$ 
    for j = 1 to N do      // Compute the total error for h[k]
      if h[k](xj)  $\neq y_j$  then error  $\leftarrow \text{error} + \mathbf{w}[j]$ 
    if error > 1/2 then break from loop
    error  $\leftarrow \min(\text{error}, 1 - \epsilon)$ 
    for j = 1 to N do      // Give more weight to the examples h[k] got wrong
      if h[k](xj) = yj then  $\mathbf{w}[j] \leftarrow \mathbf{w}[j] \cdot \text{error} / (1 - \text{error})$ 
     $\mathbf{w} \leftarrow \text{NORMALIZE}(\mathbf{w})$ 
     $\mathbf{z}[k] \leftarrow \frac{1}{2} \log((1 - \text{error}) / \text{error})$     // Give more weight to accurate h[k]
  return Function(x) :  $\sum \mathbf{z}_i \mathbf{h}_i(x)$ 
  
```

用于集成学习的自适应提升方法的 ADABOOST 变体。该算法通过对训练样例进行连续加权来生成假设。函数 WEIGHTED-MAJORITY 每次生成一个假设，该假设返回假设 **h** 中投票数最高的输出值，其投票权重为 **z**。对于回归问题或者标签为 1 和 -1 的二分类问题类，该输出即为  $\sum_k \mathbf{h}[k] \mathbf{z}[k]$ 。

# 集成学习



(a) 当  $K = 5$  时，在餐厅等待问题数据集上，提升后的决策树桩与未提升的决策树桩的性能对比。(b) 训练集和测试集上正确的比例与  $K$ （集成模型中假设的数量）的关系。值得注意的是，即使在训练精度达到1之后，即在整体模型已经准确拟合数据之后，测试集的准确率仍会略有提高

# 在线学习

- 智能体从外界接收输入  $x_j$  并预测相应的  $y_j$ , 然后知晓正确的答案, 再对  $x_{j+1}$  重复该过程, 并以此类推.
- 随机加权多数算法:** 研究每个专家过去的表现, 并选择与他们过去的表现成比例的方式给予他们相应的信任
- 初始化一个权重集合  $\{w_1, \dots, w_K\}$  全为1.
- for each* 需要解决的问题 *do*
  - 从专家处收到预测  $\{\hat{y}_1, \dots, \hat{y}_K\}$
  - 根据权重比例随机选择一个专家  $k^* : P(k) = w_k$ .
  - yield*  $\hat{y}_{k^*}$  作为专家  $k^*$  对这个问题的答案
  - 收到正确的答案  $y$ .
  - 对每一个  $\hat{y}_k \neq y$  的专家  $k$ , 更新  $w_k \leftarrow \beta w_k$
  - 归一化权重使得  $\sum_k w_k = 1$ .

其中  $\beta$  是一个常数,  $0 < \beta < 1$ , 该常数描述的是我们给每一个判断错误的专家添加惩罚的程度

- 用 **遗憾** (regret) 来衡量该算法是否成功: 与我们事后知道的拥有最佳预测记录的专家相比, 算法所犯的额外错误数量。
- $M^*$ : 最好的专家所犯的误差数量
- $M$ : 随机加权多数算法产生的误差数

$$M < \frac{M^* \ln(1/\beta) + \ln K}{1 - \beta}.$$

# 开发机器学习系统

- 问题形式化
  - 确定问题/解决该问题的哪些部分，制定损失函数
  - 度量指标
- 数据收集、评估和管理
  - 当数据有限时，数据增强的方法会有所助益
  - 有时数据的量是足够的，但被划分为不平衡类
  - 考虑数据中的离群值：与其他数据点距离很远的数据点
  - 特征工程，探索性数据分析与可视化(EDA)
- 模型选择与训练
  - 接收者操作特征(ROC)曲线
  - ROC曲线下方面积(AUC)
  - 混淆矩阵
- 信任、可解释性、可说明性
  - 源代码控制，测试，审查，监控，问责
  - 一个可说明的模型可以帮助你理解“为什么某个输入对应着某个输出”
- 操作、监护和维护
  - 监控模型在实时数据上的表现
  - 非平稳性问题——外部世界会随时间变化



# 开发机器学习系统

## 特征测试和数据测试

- (1) 特征的期望是在某个模式中捕获的。(2) 所有特征都是有益的。(3) 特征的成本不应该过高。
- (4) 特征应遵循元级要求。(5) 数据流应该有适当的隐私控制。(6) 系统可以较快地引入新的特征。
- (7) 完整测试过所有输入特征代码。

## 模型部署测试

- (1) 每个模型的规范都要经过代码审查并检入到存储库中。(2) 离线技术指标和线上服务指标应相关联。
- (3) 所有的超参数都是经过调优的。(4) 模型的过时性及其影响是已知的。(5) 简单的模型不一定更好。(6) 模型在关键的数据切片上的表现都足够好。(7) 模型已经通过了能够被包含在系统中的测试。

## 机器学习基础架构测试

- (1) 训练是可复现的。(2) 模型规范代码需经过单元测试。(3) 机器学习的整个流程需经过集成测试。(4) 在模型提供现实服务前已经检验过模型的质量。(5) 模型允许人员通过具体观测训练过程中的每个样例每一步的计算来进行调试。(6) 模型在上线生产环境前需经过灰度测试。(7) 模型可以快速且安全地进行版本回滚。

## 机器学习的监控测试

- (1) 依赖关系的改变会触发通知。(2) 训练和服务输入满足数据不变性。(3) 训练和线上服务特征计算相同值。(4) 模型不应该太陈旧。(5) 模型是数值稳定的。(6) 模型在训练速度、服务延迟、吞吐量或RAM利用率方面不应该倒退。(7) 模型在服务数据上不应该有预测质量的倒退。

# 小结

- 学习有多种形式，这取决于智能体的形式、可以进行改进的组件和可获得的反馈。
- 如果可用的反馈提供了每个样例输入的正确答案，那么我们称该学习问题为[监督学习](#)，其任务是学习某个函数  $y = h(x)$ 。当学习一个输出为连续或有序值（如[体重](#)）的函数时，我们称之为[回归](#)；当学习一个具有少量可能输出类别的函数时，我们称之为[分类](#)。
- 我们希望学习到一个函数，它不仅与现有的数据保持一致，而且与未来出现的数据也很可能一致。我们需要在与数据的一致性和假设的简单性之间做出权衡。
- 一个学习算法的表现可以通过[学习曲线](#)进行可视化，它所给出的是模型在[测试集](#)上的预测准确率关于[训练集](#)大小的函数关系。
- 当有多个模型可供选择时，[模型选择](#)可以通过在验证集上进行[交叉验证](#)来选出较好的超参数。一旦超参数被选取，我们就可以使用所有的训练数据来构建最好的模型。
- 不是所有的错误都是平等的。[损失函数](#)可以告诉我们一个错误的严重程度，那么我们的目标则是最小化验证集上的损失函数。
- [计算学习理论](#)分析了归纳式学习的样本复杂性和计算复杂性。在假设空间的表达能力与学习的容易程度之间应当有一个权衡。



# 小结

- 决策树可以表示任何一个布尔函数。基于信息增益的启发式算法为寻找一个简单且一致的决策树提供了一个有效途径。
- 线性回归是一个被广泛运用的模型，它的最优参数值可以被精确地计算出来或者通过梯度下降搜索找到，其中梯度下降搜索是一种可以用于解决不存在闭式解的模型的技术。
- 一个带有硬阈值的线性分类器——也被称为感知机——可以通过简单的参数更新规则训练，并且能拟合线性可分的数据。对于线性不可分的数据，这个更新规则将不能收敛。
- 逻辑斯谛回归将感知机所用的硬阈值替换为由逻辑斯谛函数定义的软阈值。即使是带噪声的线性不可分数据集，梯度下降也能在该模型中表现得很好。
- 非参数模型使用所有的数据来单独做每一个预测，而不是试图用若干个参数总结出数据中的信息。例如最近邻与局部加权回归。
- 支持向量机通过寻找带有最大边距的线性分离器来改进分类的泛化表现。即使原数据不是线性可分的，核方法也可以隐式地将输入数据投影到可能存线性分离器的高维空间中。
- 自助聚合法与自适应提升法这样的集成方法通常比单独的方法表现得更好。在在线学习中，即使数据的分布不断变化，我们也可以通过聚集专家们的意见来达到任意程度接近最好的专家的表现。
- 构建一个良好的机器学习模型要求我们在从数据管理到模型选择和优化，再到持续性维护的整个开发过程中都有一定经验。

Q&A

Thanks!

